



Deep Learning

Lecture on cnns
UG-3

Convolution

1D (continuous, discrete) :

$$f * g(x) = \int_{\alpha=-\infty}^{\infty} f(\alpha)g(x-\alpha)d\alpha$$

$$= \sum_{\alpha=0}^{N-1} f(\alpha)g(x-\alpha)$$

Input

Kernel

Output is
sometimes called
Feature map

2D (continuous, discrete) :

$$f * g(x, y) = \int_{\alpha=-\infty}^{\infty} \int_{\beta=-\infty}^{\infty} f(\alpha, \beta)g(x-\alpha, y-\beta)d\alpha d\beta$$

$$= \sum_{\alpha=0}^{N-1} \sum_{\beta=0}^{N-1} f(\alpha, \beta)g(x-\alpha, y-\beta)$$

Convolution Properties

- Commutative:

$$f * g = g * f$$

- Associative:

$$(f * g) * h = f * (g * h)$$

- Homogeneous:

$$f * (\lambda g) = \lambda f * g$$

- Additive (Distributive):

$$f * (g + h) = f * g + f * h$$

- Shift-Invariant

$$f * g(x - x_0, y - y_0) = (f * g)(x - x_0, y - y_0)$$

The convolution operation

Input Matrix				Kernel			Result	
45	12	5	17	0	-1	0	-45	12
22	10	35	6	-1	5	-1	22	10
88	26	51	19	0	-1	0		
9	77	42	3					

$$\begin{aligned}
 &45 * 0 + 12 * (-1) + 5 * 0 \\
 &+ 22 * (-1) + 10 * 5 + 35 * (-1) \\
 &+ 88 * 0 + 26 * (-1) + 51 * 0
 \end{aligned}$$

= -45

45	12	5	17	0	-1	0	-45	103
22	10	35	6	-1	5	-1	22	10
88	26	51	19	0	-1	0		
9	77	42	3					

45	12	5	17	0	-1	0	-45	103
22	10	35	6	-1	5	-1	-96	10
88	26	51	19	0	-1	0		
9	77	42	3					

45	12	5	17	0	-1	0	-45	103
22	10	35	6	-1	5	-1	-176	133
88	26	51	19	0	-1	0		
9	77	42	3					

Convolution in digital image processing

<i>Original</i>	<i>Gaussian Blur</i>	<i>Sharpen</i>	<i>Edge Detection</i>
$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$
			

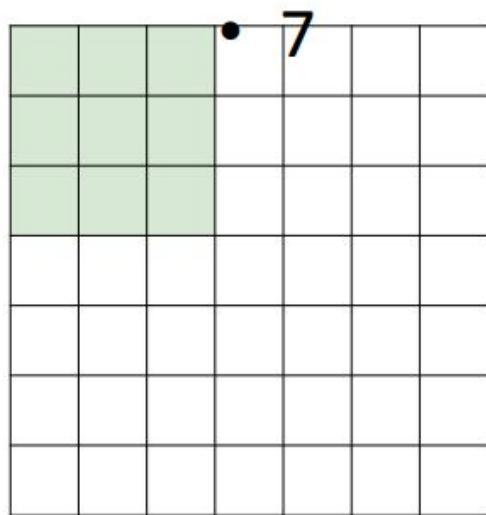
Traditional kernels are

- fixed,
- task-agnostic, and
- linear,

whereas CNN-learned kernels are data-driven, task-specific, adaptive, and composable across layers.

Convolutions: More detail

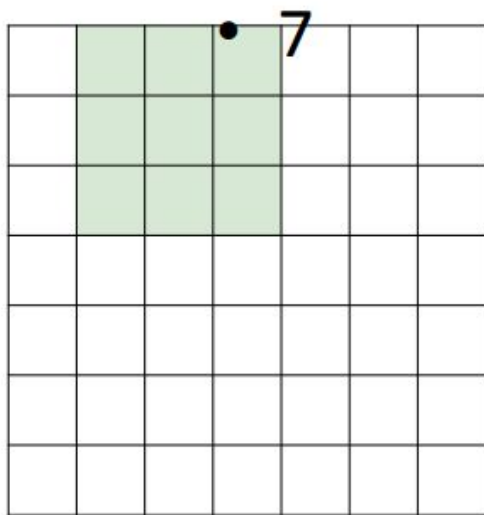
A closer look at spatial dimensions:



- 7x7 input (spatially)
assume 3x3 filter

Convolutions: More detail

A closer look at spatial dimensions:

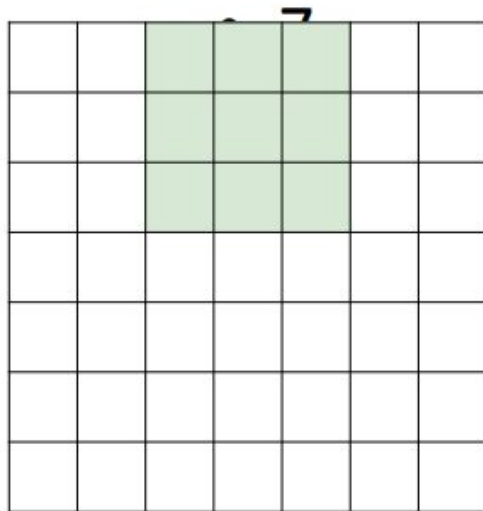


- 7x7 input (spatially)
assume 3x3 filter

• 7

Convolutions: More detail

A closer look at spatial dimensions:

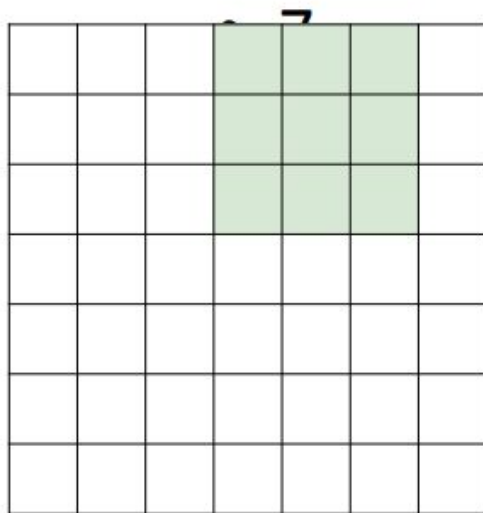


- 7x7 input
(spatially)
assume 3x3
filter

• 7

Convolutions: More detail

A closer look at spatial dimensions:

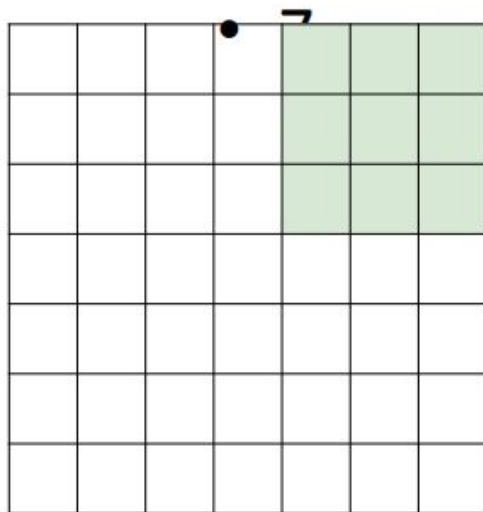


- 7x7 input
(spatially)
assume 3x3
filter

• 7

Convolutions: More detail

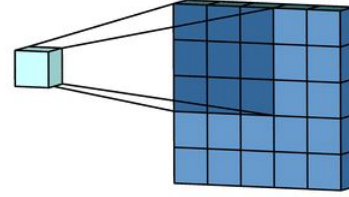
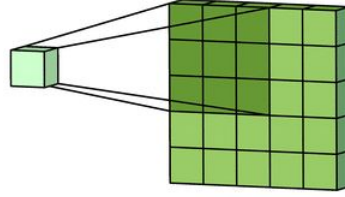
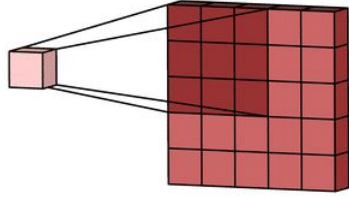
A closer look at spatial dimensions:

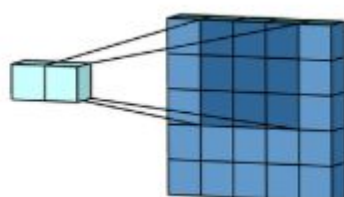
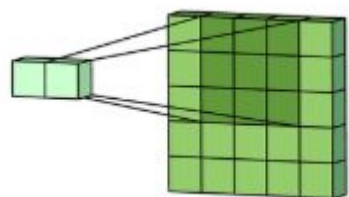
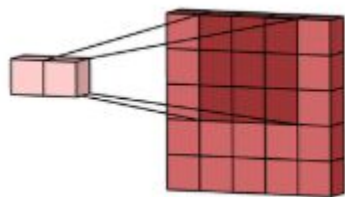
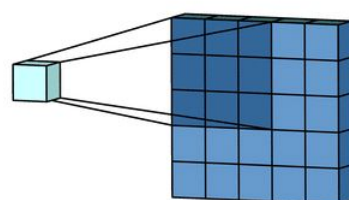
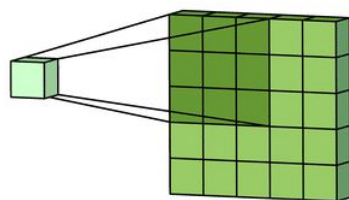
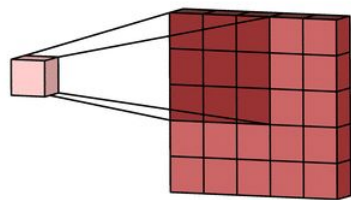


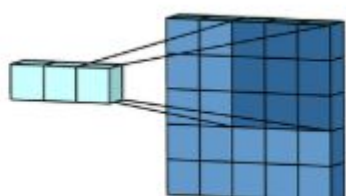
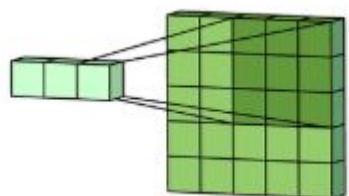
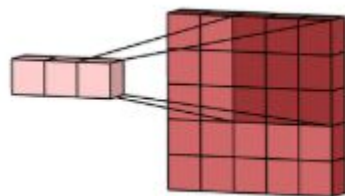
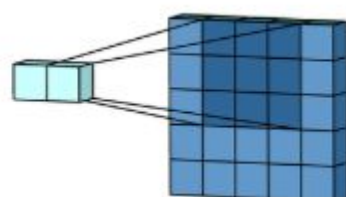
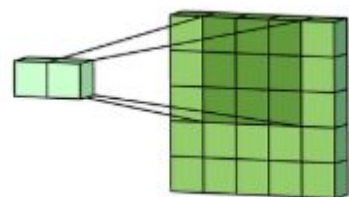
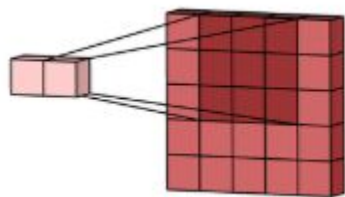
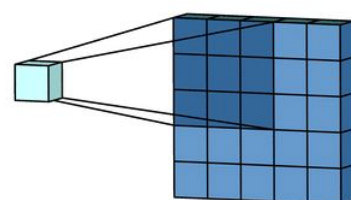
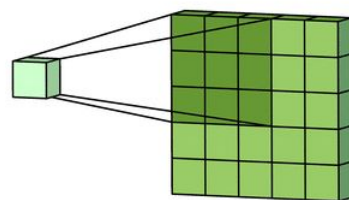
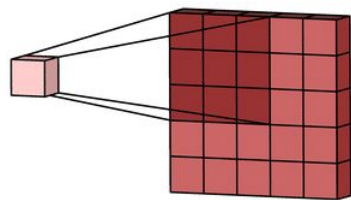
- 7x7 input (spatially)
assume 3x3 filter

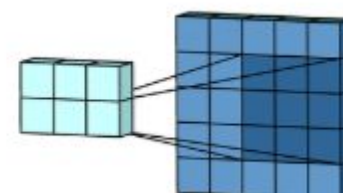
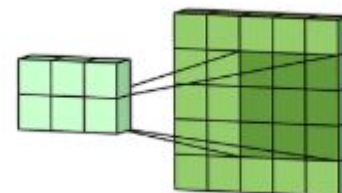
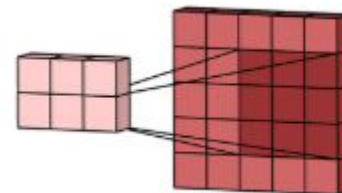
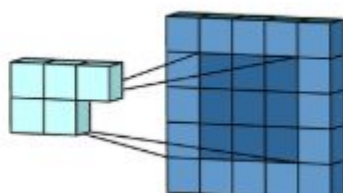
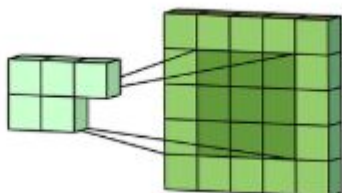
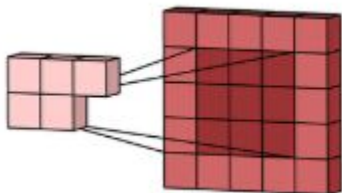
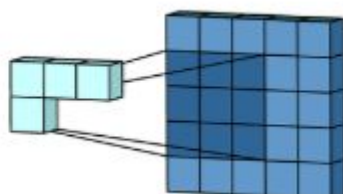
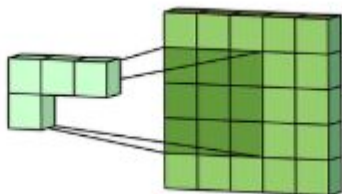
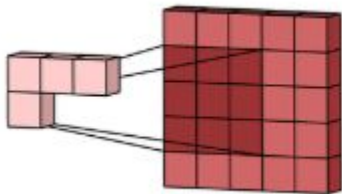
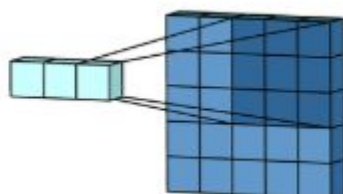
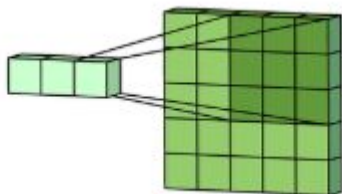
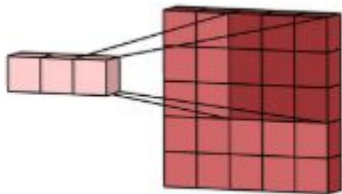
7 => 5x5 output

Convolution in multi channel inputs

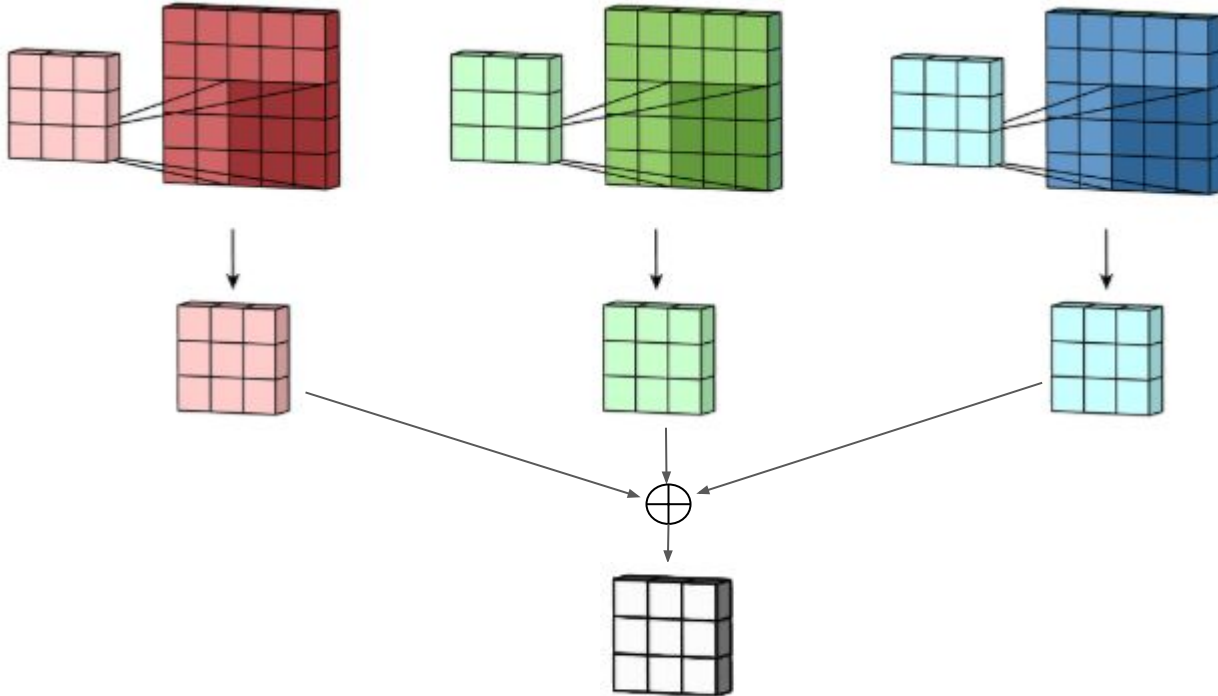






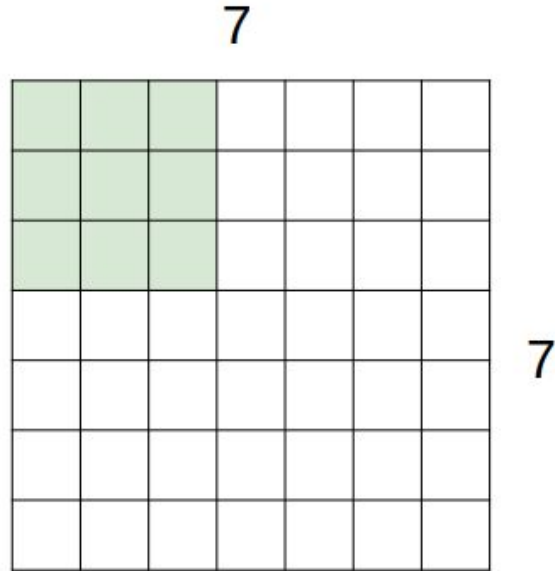


5x5x3 (RGB) image is convolved with 3x3x3 kernel to get a 3x3x1



Convolutions: More detail Stride

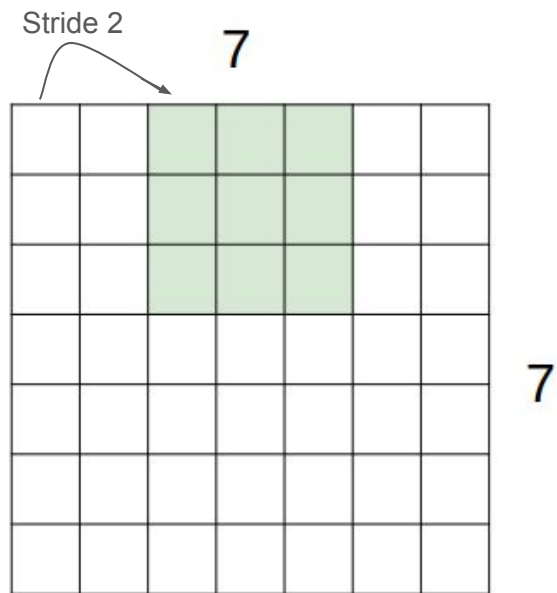
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

Convolutions: More detail

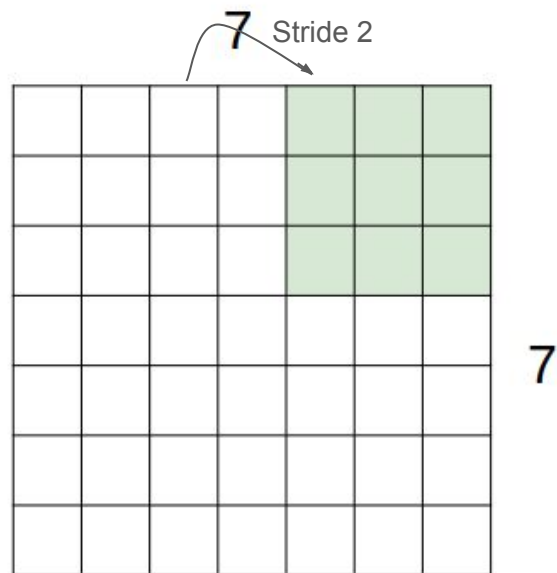
A closer look at spatial dimensions:



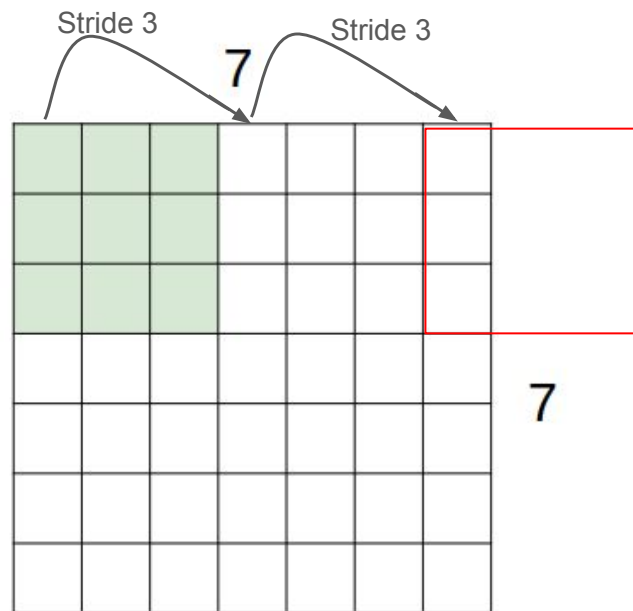
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

Convolutions: More detail

A closer look at spatial dimensions:

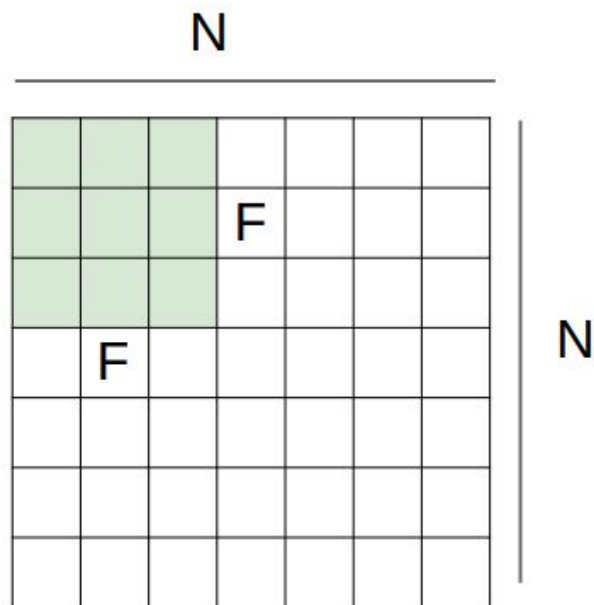


7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

doesn't fit!
cannot apply 3x3 filter on
7x7 input with stride 3.



Output size:

$$(N - F) / \text{stride} + 1$$

e.g. $N = 7, F = 3$:

$$\text{stride } 1 \Rightarrow (7 - 3) / 1 + 1 = 5$$

$$\text{stride } 2 \Rightarrow (7 - 3) / 2 + 1 = 3$$

$$\text{stride } 3 \Rightarrow (7 - 3) / 3 + 1 = 2.33 \text{ :}\backslash$$

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

(recall:)

$$(N - F) / \text{stride} + 1$$

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

in general, common to see CONV layers with stride 1, filters of size $F \times F$, and zero-padding with $(F-1)/2$. (will preserve size spatially)

e.g. $F = 3 \Rightarrow$ zero pad with 1

$F = 5 \Rightarrow$ zero pad with 2

$F = 7 \Rightarrow$ zero pad with 3

$$(N + 2*\text{padding} - F) / \text{stride} + 1$$

Examples

Input volume: **32x32x3**

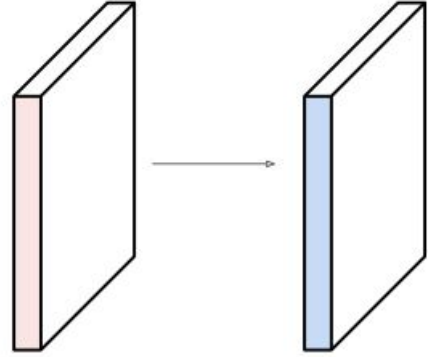
10 **5x5** filters with stride **1**, pad **2**

↙
(Depth:3 is inherent => 5x5x3)

Output volume size:

$(32+2*2-5)/1+1 = 32$ spatially, so

32x32x10



- Accepts a volume of size $W1 \times H1 \times D1$
- Requires four hyperparameters:
 - Number of filters K
 - their spatial extent F
 - the stride S
 - the amount of zero padding P
- Produces a volume of size $W2 \times H2 \times D2$
 - $W2 = (W1 - F + 2P) / S + 1$
 - $H2 = (H1 - F + 2P) / S + 1$
 - $D2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D1$ weights per filter, for a total of $(F \cdot F \cdot D1) \cdot K$ weights and K biases.
- In the output volume, the d -th depth slice (of size $W2 \times H2$) is the result of performing a valid convolution of the d -th filter over the input volume with a stride of S , and then offset by d -th bias.

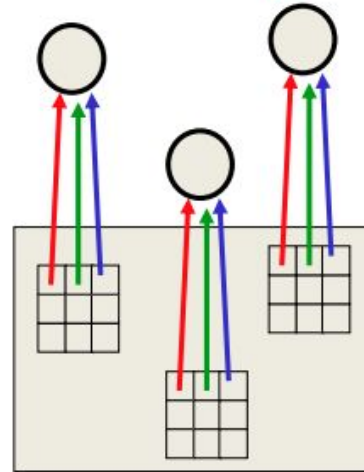
Data flow in a Convolutional layer

The replicated feature approach

Why same kernel is used across all positions in an image/feature map?

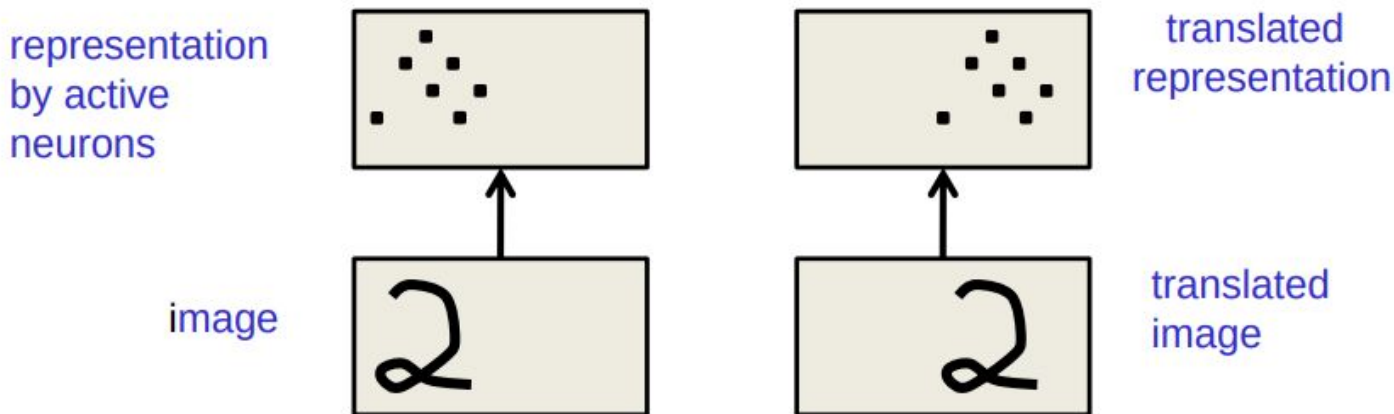
- Use many different copies of the same feature detector with different positions.
 - Could also replicate across scale and orientation (*tricky and expensive*)
 - Replication greatly reduces the number of free parameters to be learned.
- Use several different feature types, each with its own map of replicated detectors.
 - Allows each patch of image to be represented in several ways.

The red connections all have the same weight.



What does replicating the feature detectors achieve?

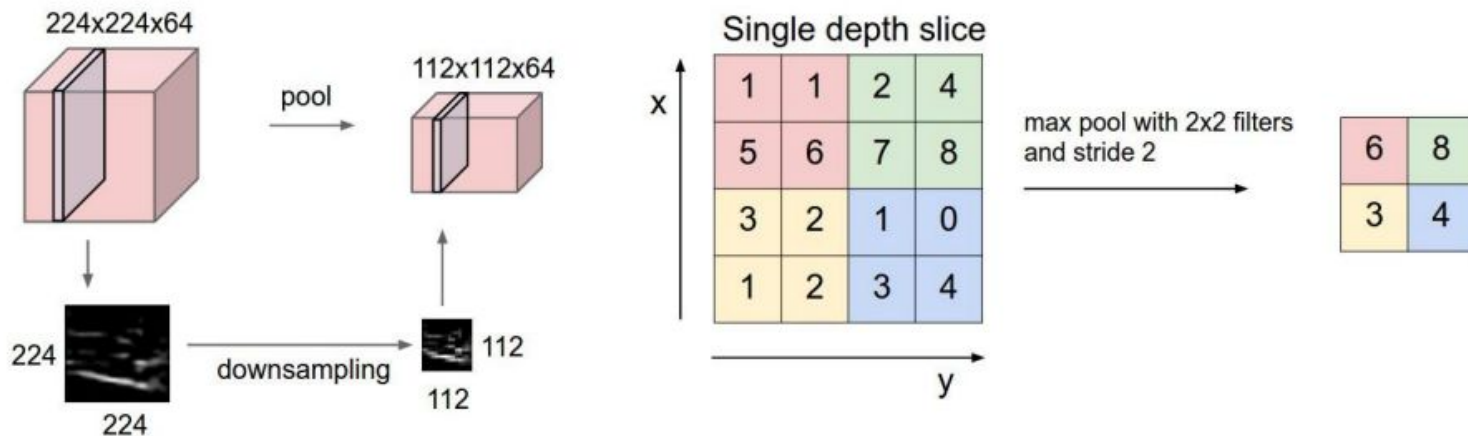
- **Equivariant activities:** Replicated features do **not** make the neural activities invariant to translation. The activities are equivariant.



- **Invariant knowledge:** If a feature is useful in some locations during training, detectors for that feature will be available in all locations during testing.

3. Spatial Pooling

- Sum or max over non-overlapping / overlapping regions
- Role of pooling:
 - Invariance to small transformations
 - Larger receptive fields (neurons see more of input)

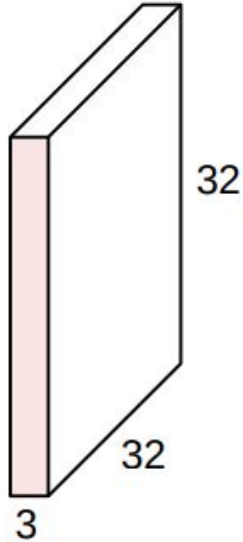


General pooling layer

- Accepts a volume of size $W1 \times H1 \times D1$
- Requires two hyperparameters:
 - their spatial extent F
 - the stride S
- Produces a volume of size $W2 \times H2 \times D2$ where:
 - $W2 = (W1 - F) / S + 1$
 - $H2 = (H1 - F) / S + 1$
 - $D2 = D1$
- Introduces zero parameters
- Other pooling functions: Average pooling, L2-norm pooling

Preview of CNN

32x32x3 image

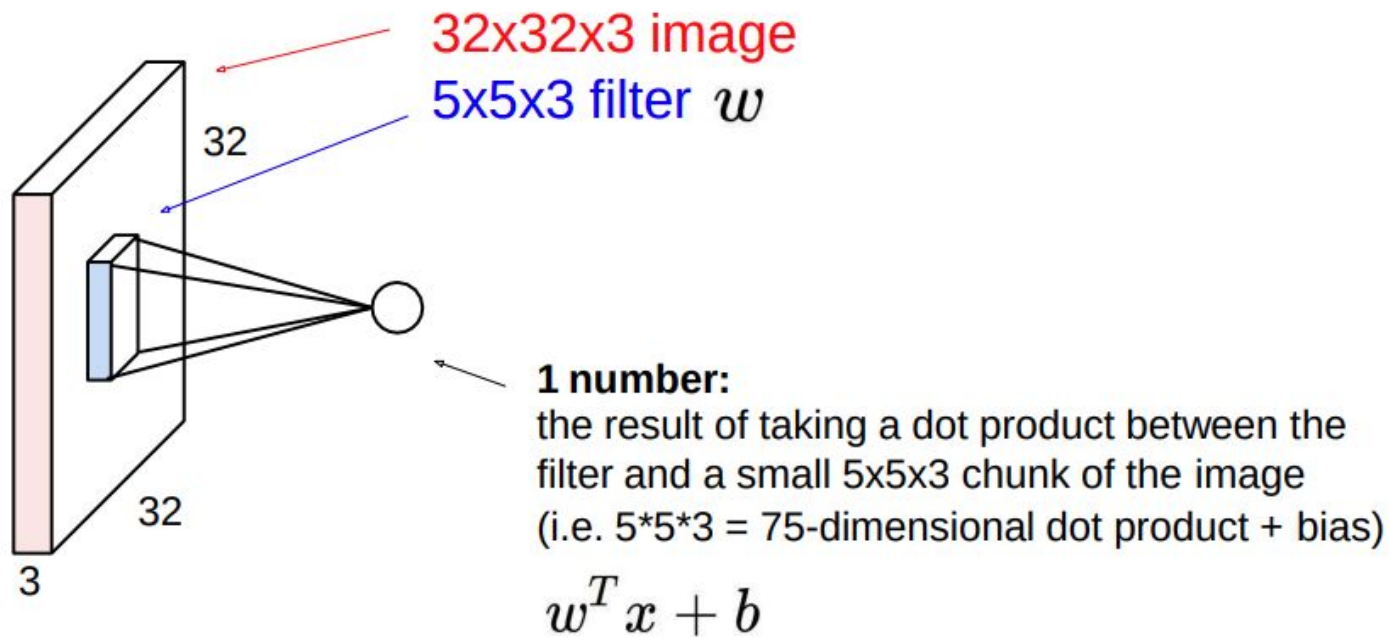


5x5x3 filter

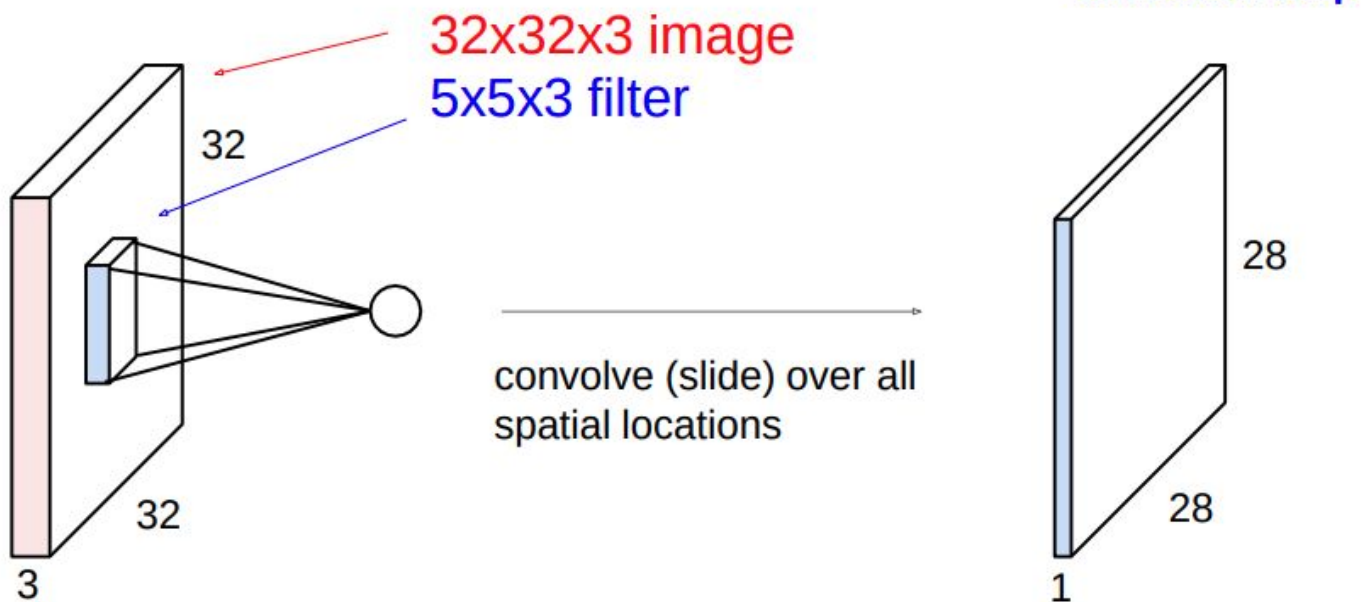


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

Convolution Layer

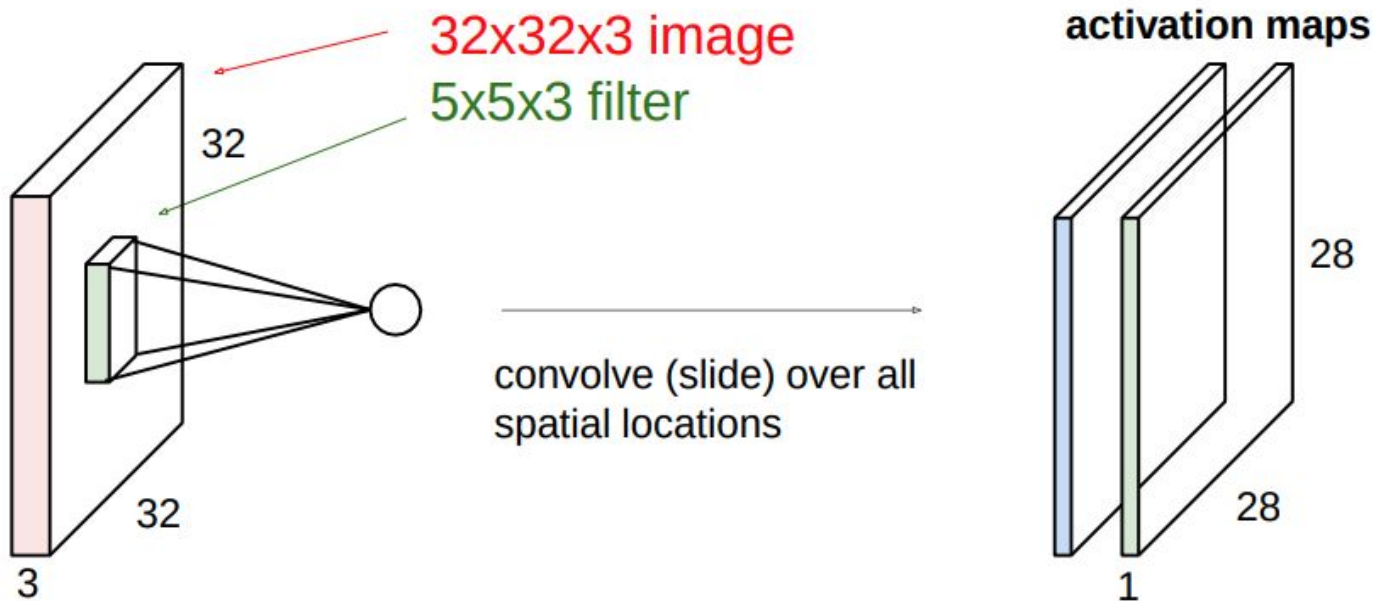


Convolution Layer

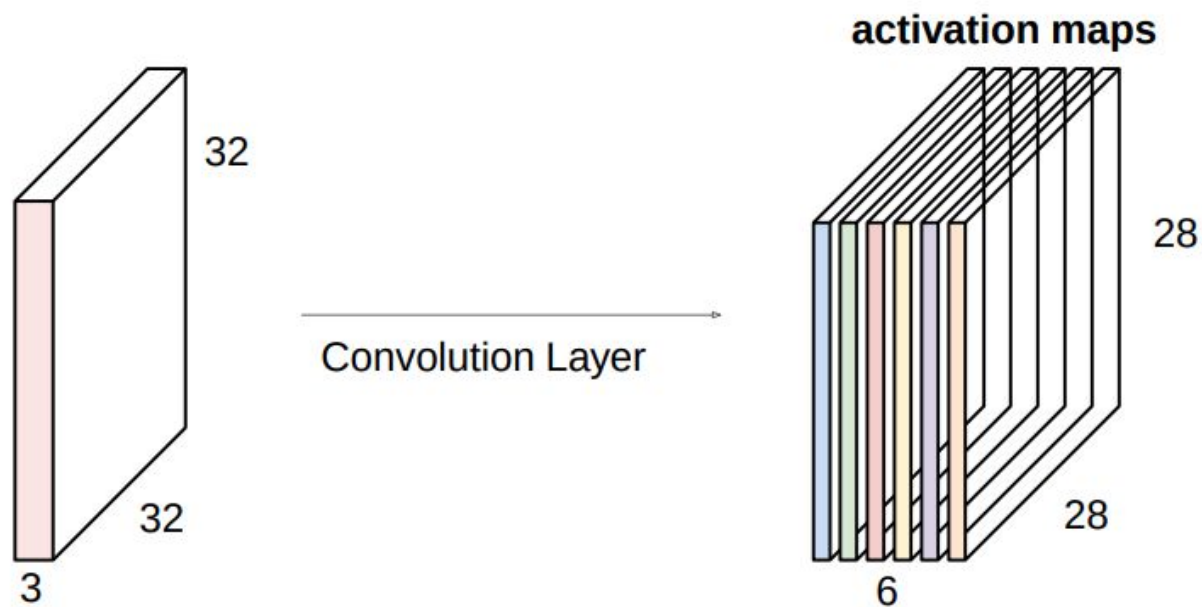


Convolution Layer

consider a second, **green** filter

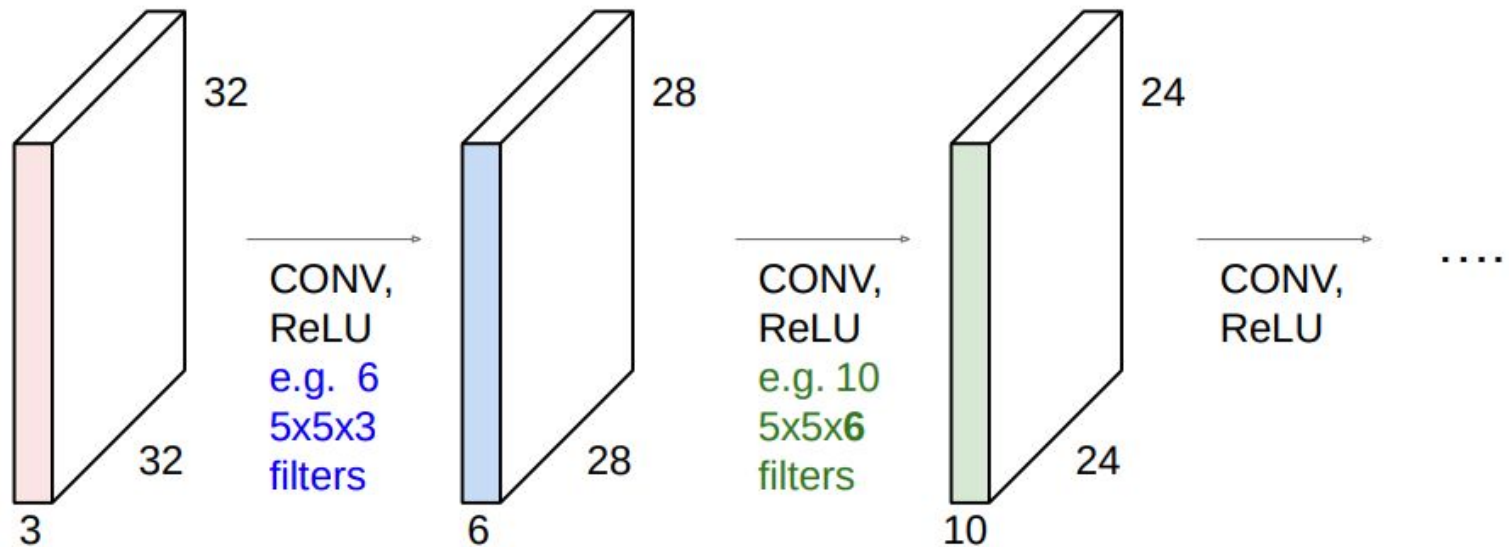


For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

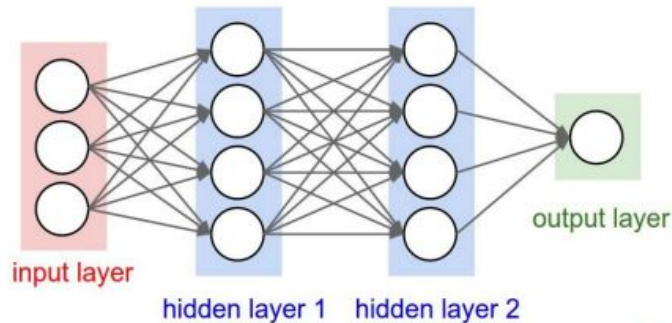


We stack these up to get a “new image” of size 28x28x6!

Preview: ConvNet is a sequence of Convolutional Layers, interspersed with activation functions

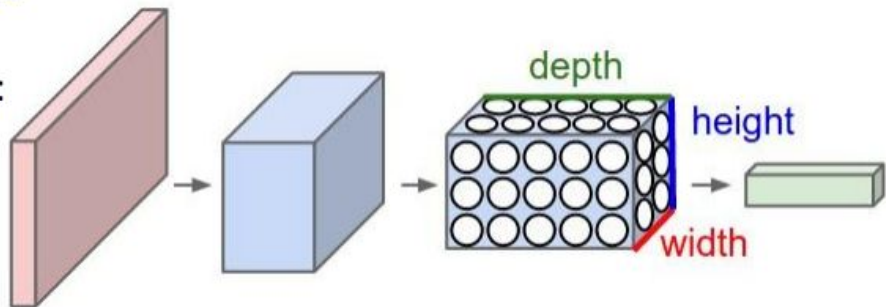


Convnets



Layers used to build ConvNets:

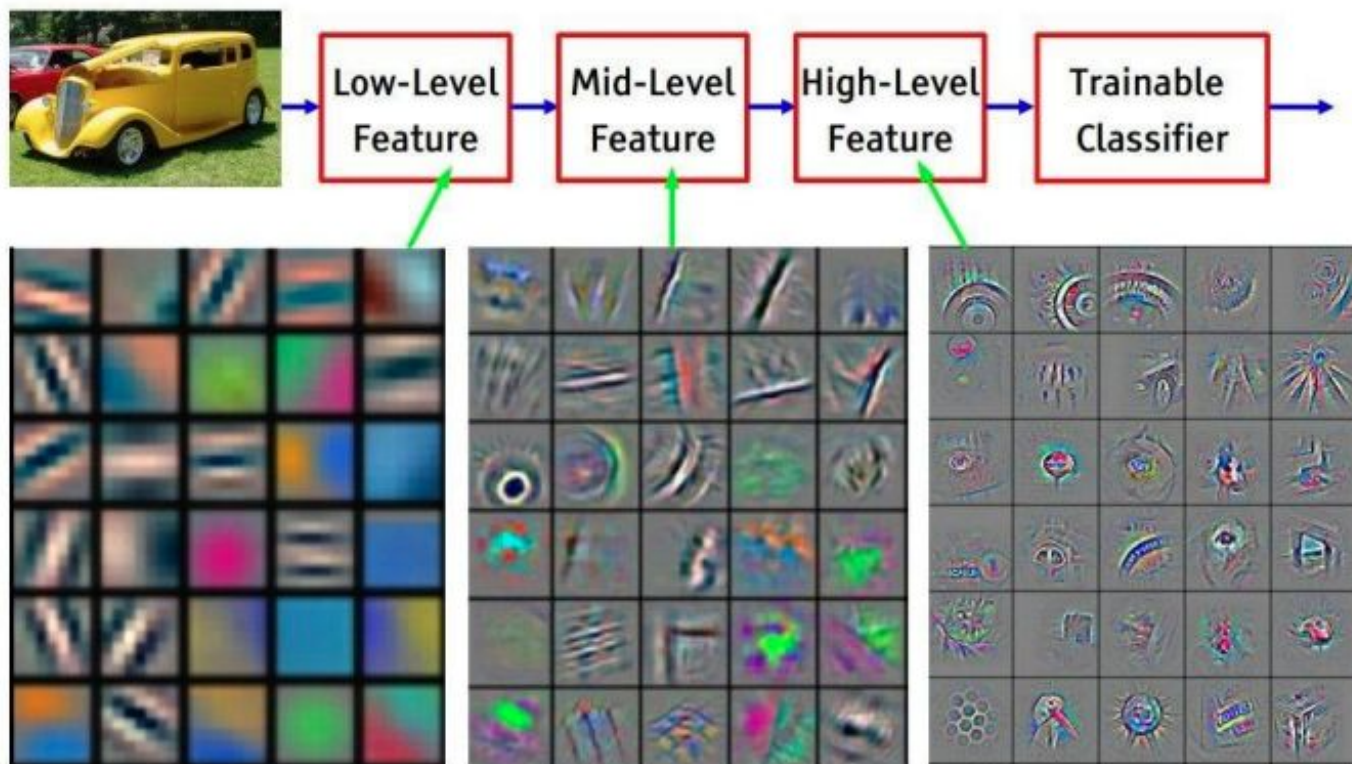
- a stacked sequence of layers. 3 main types
- Convolutional Layer, Pooling Layer, and Fully-Connected Layer



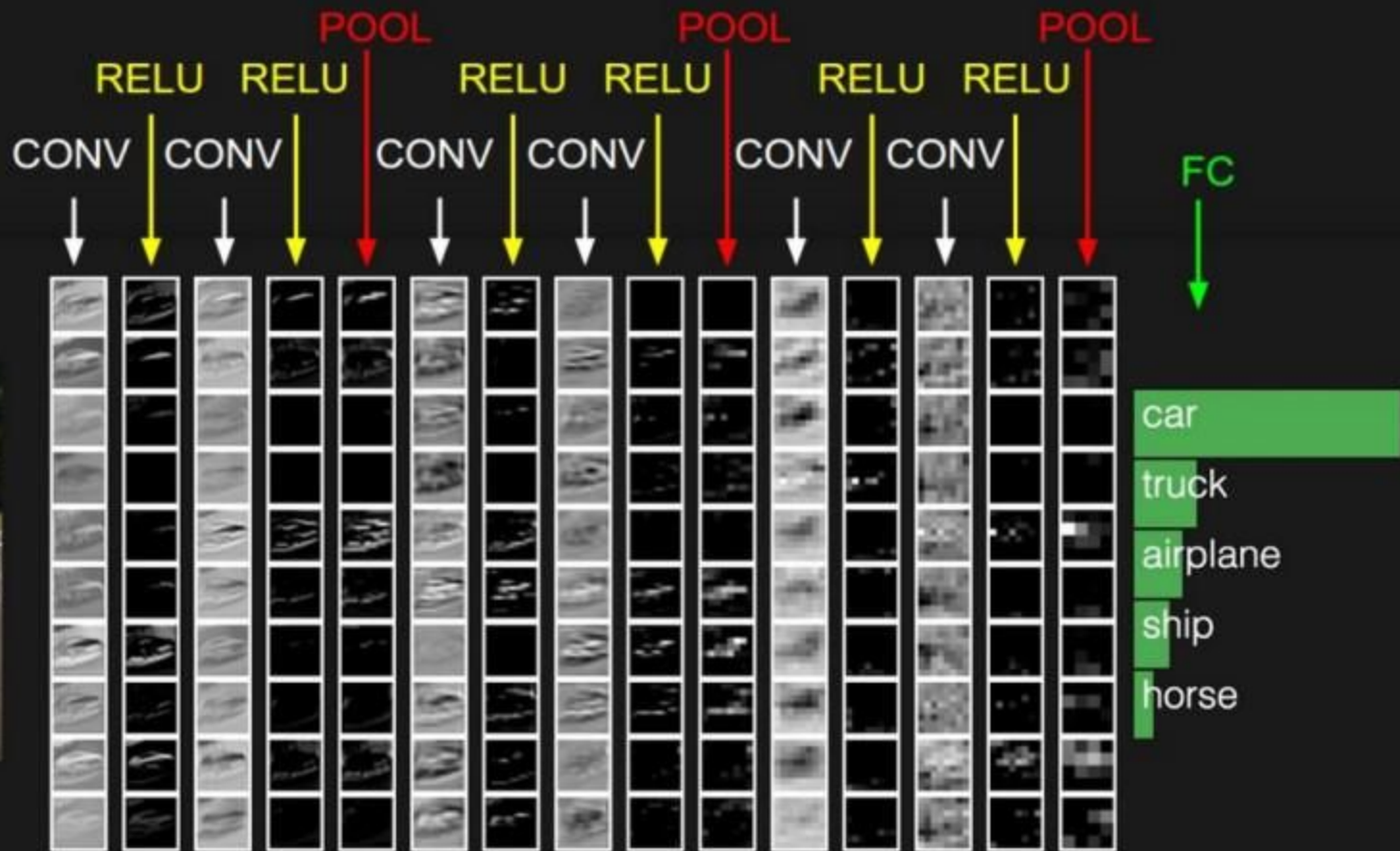
- every layer of a ConvNet transforms one volume of activations to another through a differentiable function.

INPUT -> [[CONV -> RELU]*N -> POOL?]*M -> [FC -> RELU]*K -> FC

- $N \geq 0$ (and usually $N \leq 3$), $M \geq 0$, $K \geq 0$

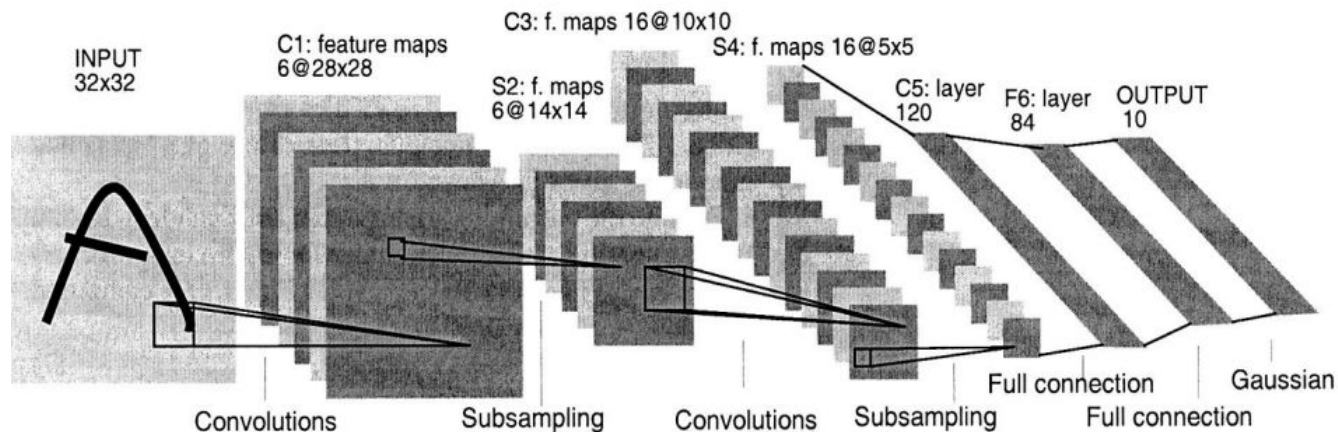


Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]



The architecture of LeNet5

1998, Yann LeCun



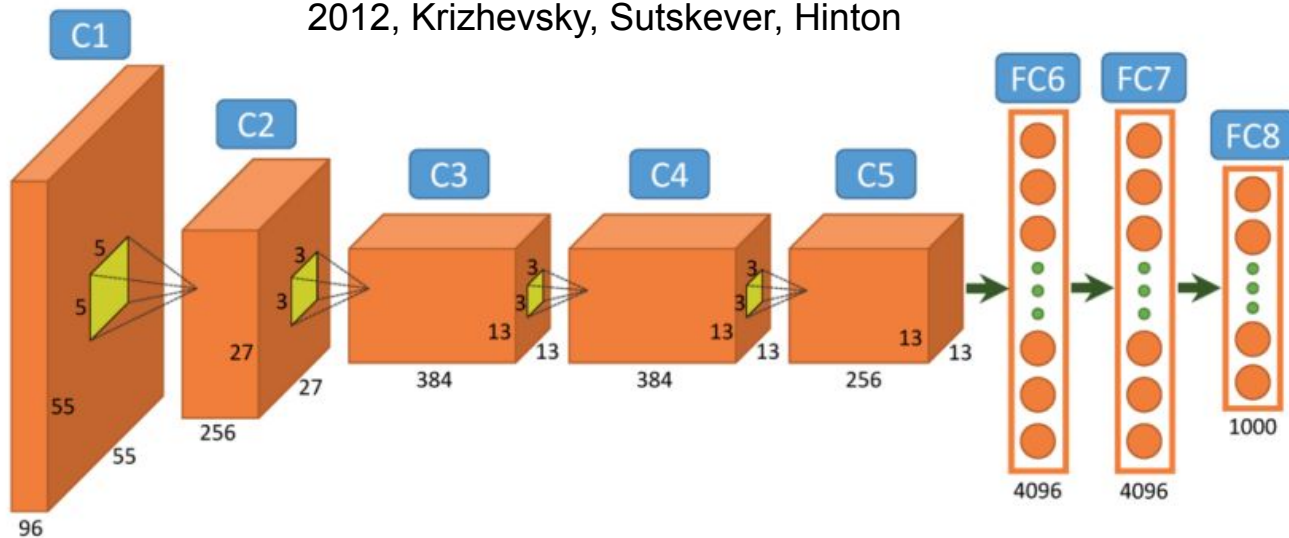
First successful convolutional neural network for handwritten digit recognition (MNIST).
Reduce parameters compared to fully connected nets, exploit spatial locality.

Architecture:

- Stacked **convolutions + pooling** followed by fully connected layers.
- Introduced weight sharing, local receptive fields.

A Common Architecture: AlexNet

2012, Krizhevsky, Sutskever, Hinton



Winning **ImageNet 2012** by a huge margin (15.3% top-5 error vs ~26% previous).

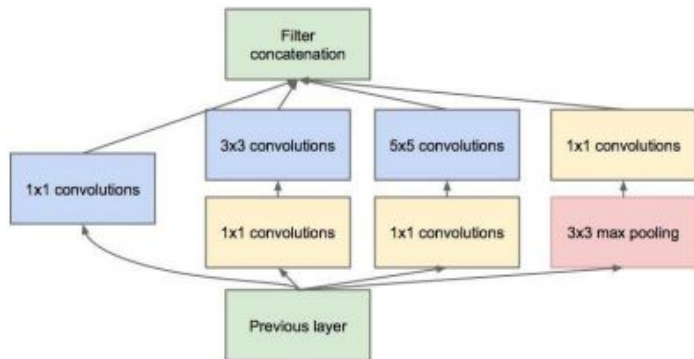
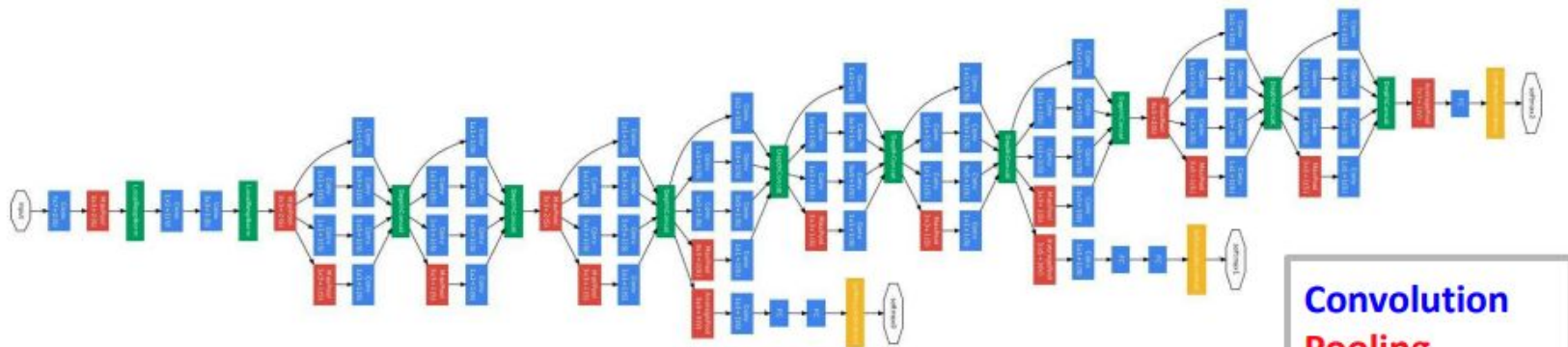
Showed CNNs + GPUs could scale to large datasets and crush hand-crafted features (SIFT, HOG).

Architecture:

- 8 layers (5 conv, 3 fully connected).
- Introduced **ReLU** (faster training than sigmoid/tanh).
- Used **Dropout** (to reduce overfitting).
- Trained on **GPUs** (massively sped up deep learning).

GoogLeNet

[Szegedy et al., 2014]



Inception module

Inception modules: parallel convs (1×1 , 3×3 , 5×5) + pooling $\rightarrow$ concatenated.
Used **1×1 convolutions** as **bottleneck layers** to reduce computation.

ILSVRC 2014 winner (6.7% top 5 error)

Large Scale Visual Recognition Challenge

ResNet

2015, Kaiming He et al

Showed **depth = power**, but training very deep nets is hard.

Residual learning solved optimization difficulties.

- Introduced **Residual connections (skip connections)**: alleviates **vanishing gradient problem**, enabling ultra-deep training.
- Up to **152 layers** deep., improved accuracy.
- Winner ILSVRC 2015 with 3.5% (top-5) beat human-level performance (5%) in object recognition.

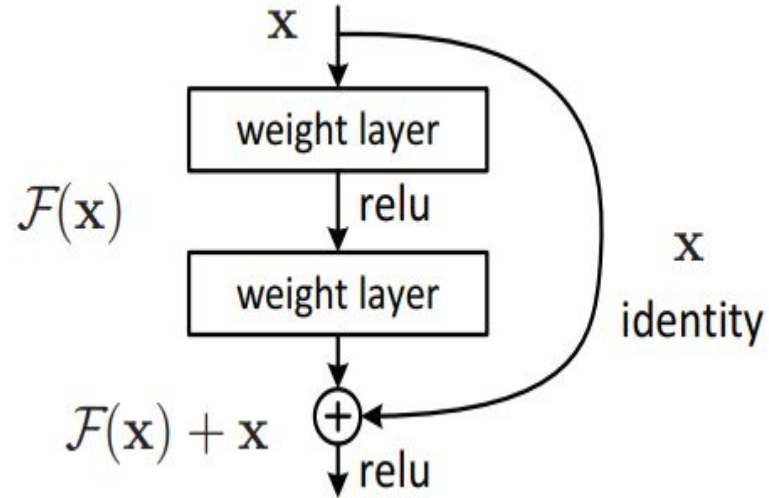


Image Segmentation

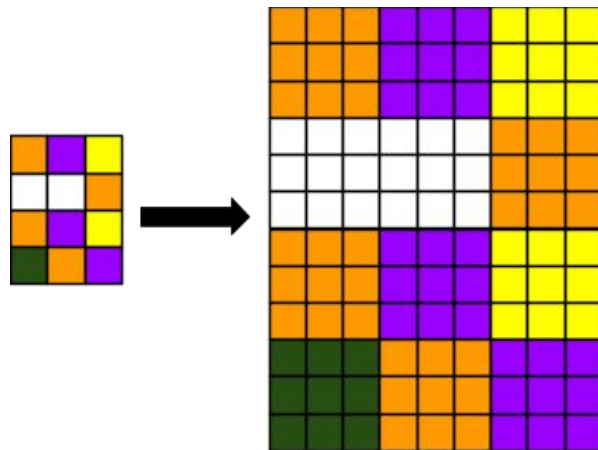
Upsampling: Resize feature map to larger size

- **Nearest neighbor:** copy nearest pixel value.
- **Bilinear:** weighted average of 4 nearest pixels.

Transposed Convolution (Deconvolution):

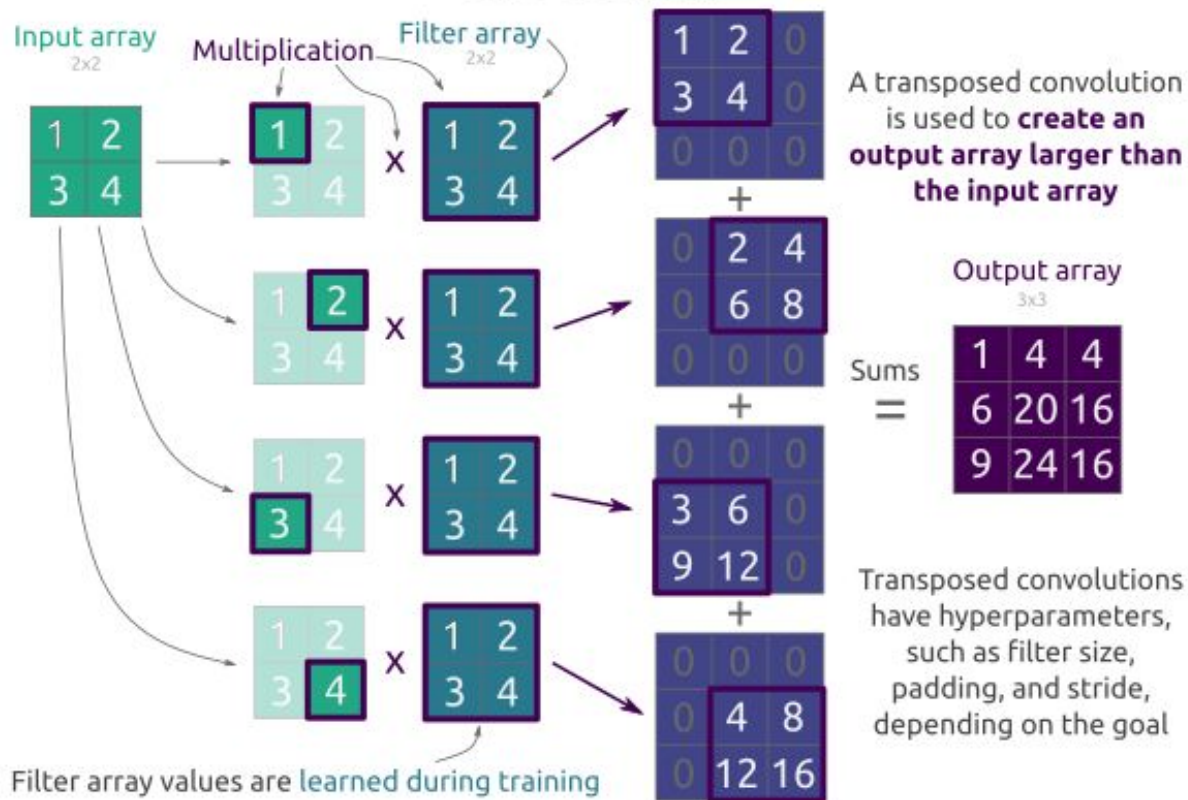
$$\text{Convolution} = (N - F + 2P) / S + 1$$

$$\text{Deconv/TC} = (N - 1)S - 2P + F$$

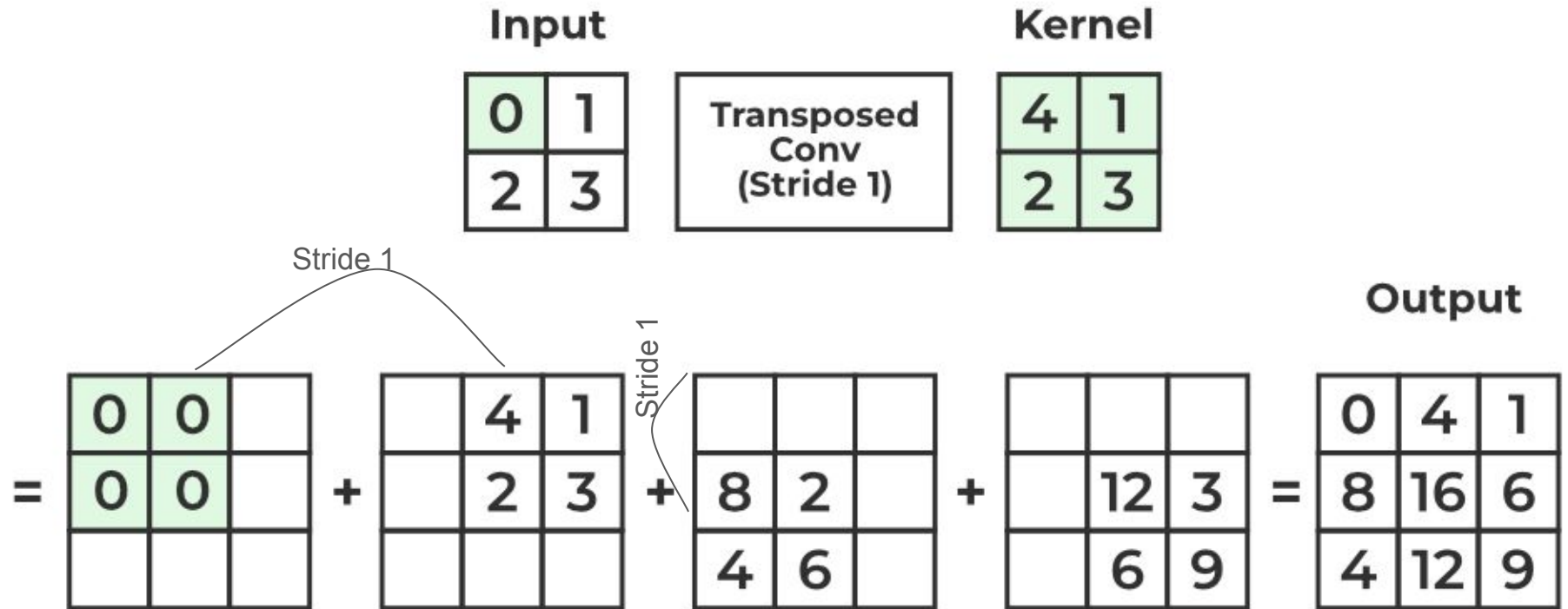


Transposed Convolutions

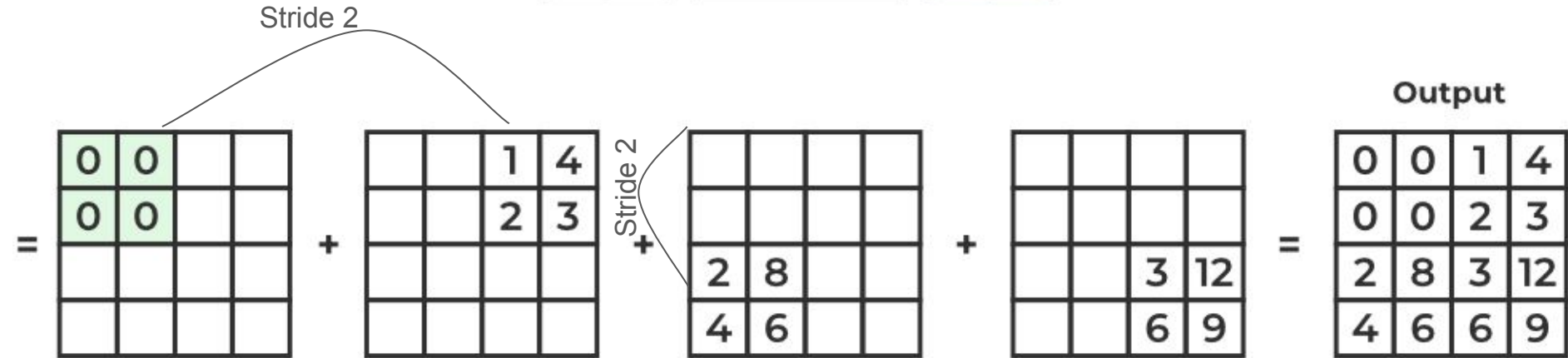
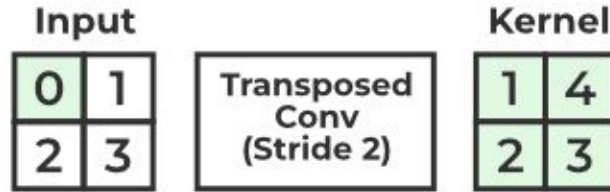
"Smart" upsampling



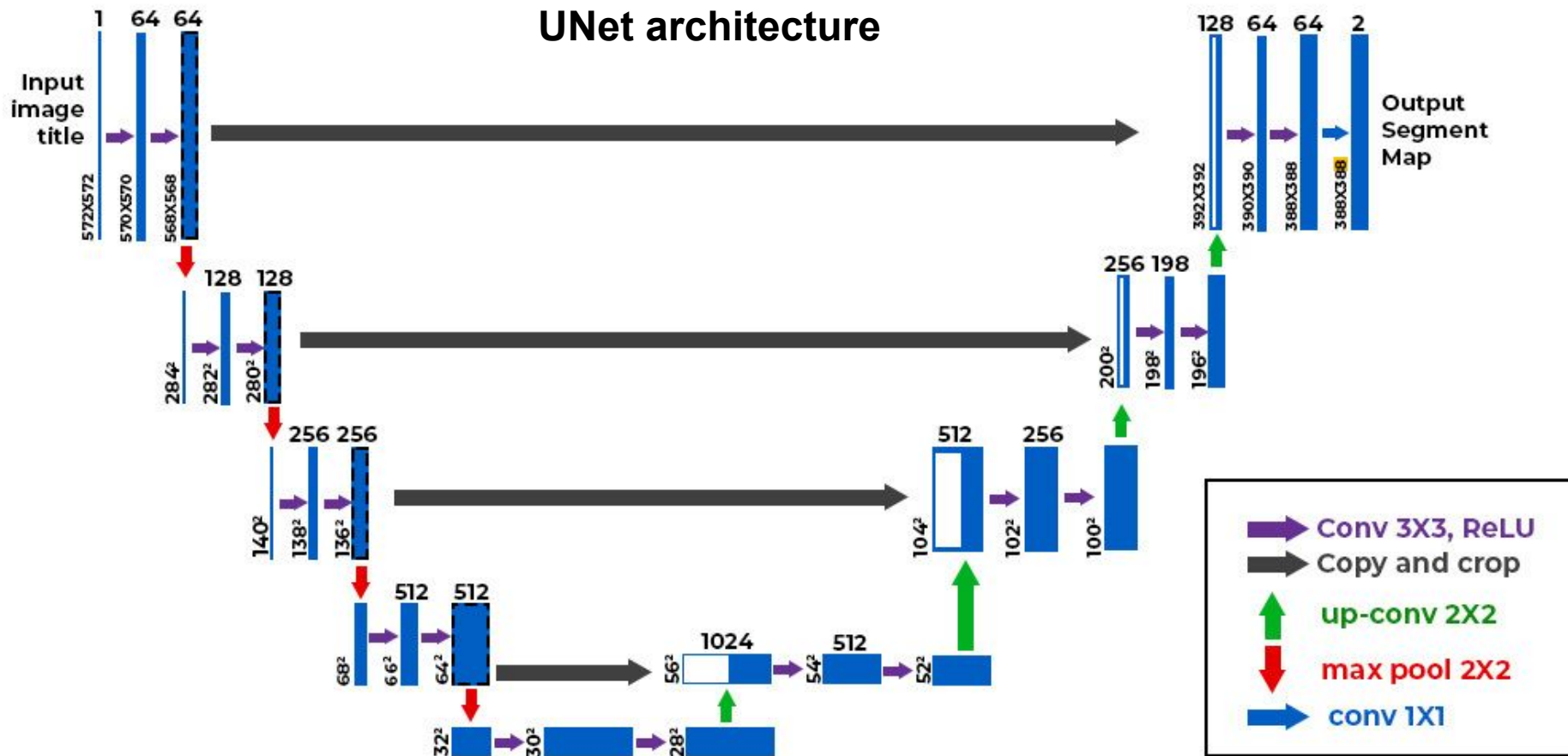
$$\text{Out_size} = (N-1)S - 2P + F = (2-1)*1 - 0 + 2 = 3$$



$$\text{Out_size} = (N-1)S - 2P + F = (2-1)*2 - 0 + 2 = 4$$



UNet architecture



Object recognition in images

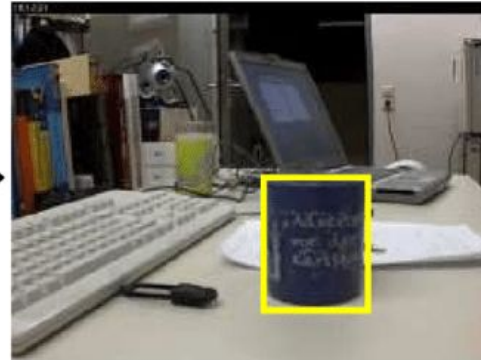
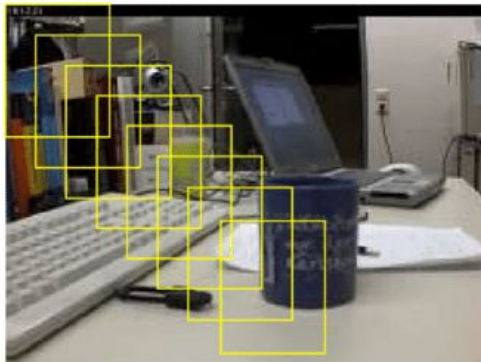
Traditional Object recognition process

Suppose the image is 1000×600 pixels.

Use a 64×64 detection window, stride 8 pixels.

- Horizontal positions $\approx (1000-64)/8 \approx 117$
- Vertical positions $\approx (600-64)/8 \approx 67$
- Windows at one scale $\approx 117 \times 67 \approx \mathbf{7,800}$.

Now test 10 different window sizes/scales $\rightarrow \sim \mathbf{78,000}$ windows.

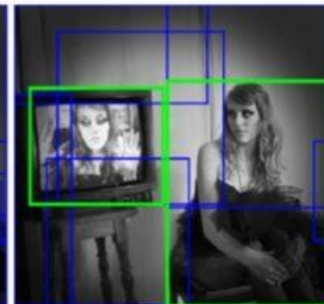
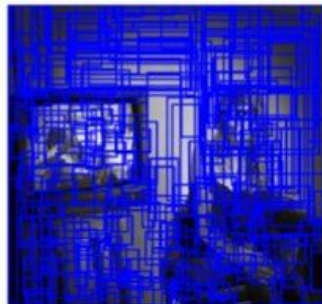


R-CNN (Regions with CNN features)

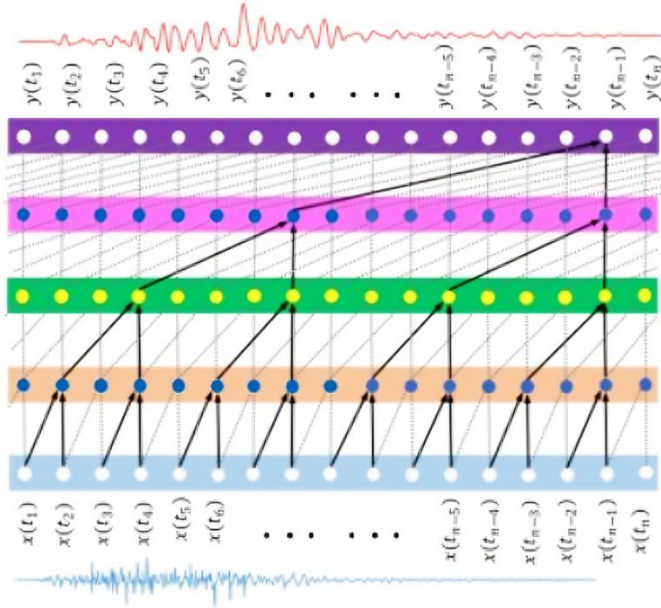
- Oversegment the image into superpixels.
- Merge neighboring superpixels that look similar (color, texture, size, fill).
- Every merge $\rightarrow$ a candidate box.
- After deduplication you keep ≈ 2000 high-quality proposals.



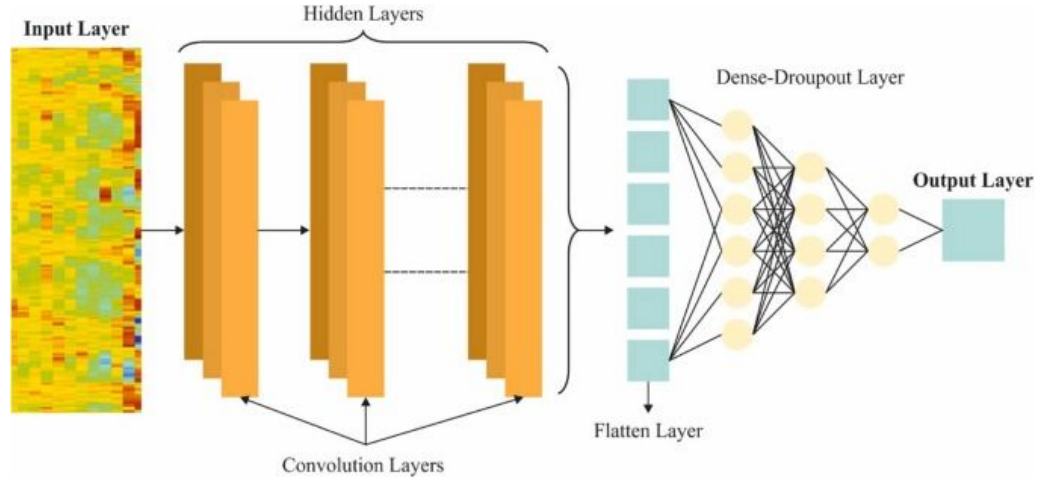
Input Image



1D CNN



1D CNNs on raw wav samples



1D CNNs on processed speech vectors

Back Propagation in CNNs

Chain rule review:

$$f(x,y,z) = (x + y)z$$

$$\frac{\partial q}{\partial x} = 1 \quad \frac{\partial q}{\partial y} = 1$$

Let $q=x+y$

$f=q*z$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} * \frac{\partial q}{\partial x} = -4$$

Local gradients

$$x = -2$$

$$\frac{\partial f}{\partial x} = -4$$

$$y = 5$$

$$\frac{\partial f}{\partial y} = -4$$

$$z = -4$$

$$\frac{\partial f}{\partial z} = 3$$

f

$$+$$

$$q = 3$$

$$\frac{\partial f}{\partial q} = -4$$

$$f = q * z$$

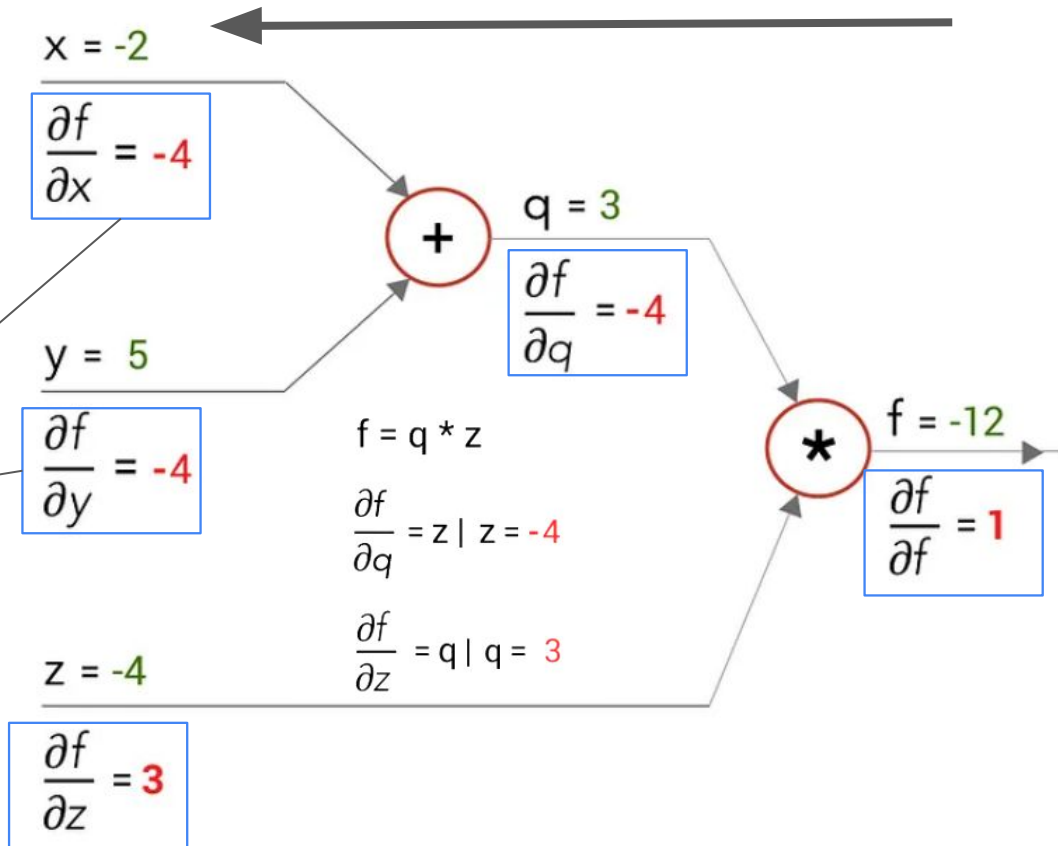
$$\frac{\partial f}{\partial q} = z \mid z = -4$$

$$\frac{\partial f}{\partial z} = q \mid q = 3$$

$$*$$

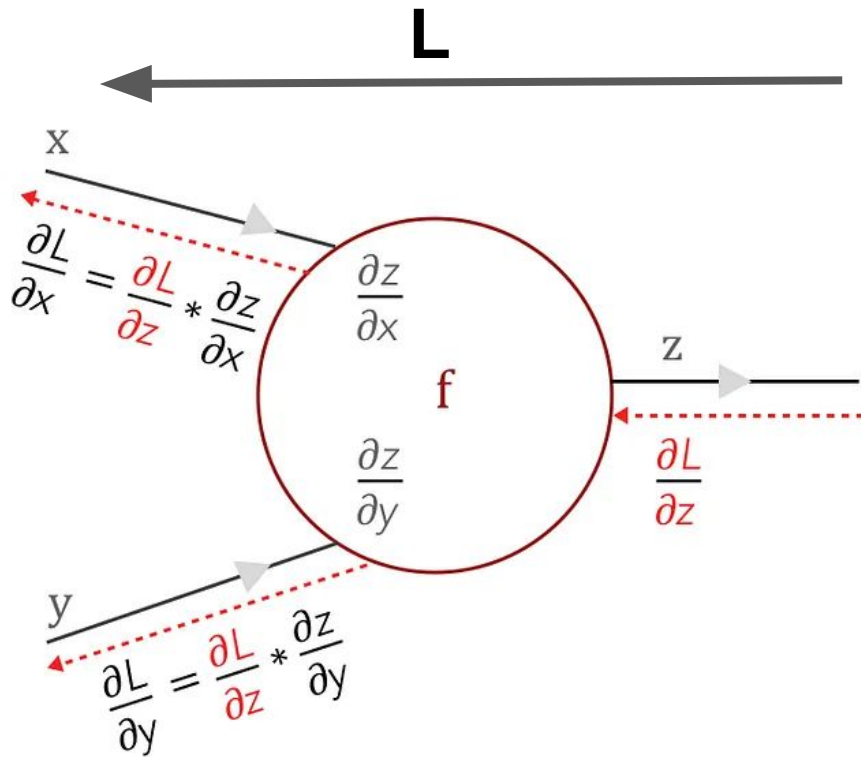
$$f = -12$$

$$\frac{\partial f}{\partial f} = 1$$

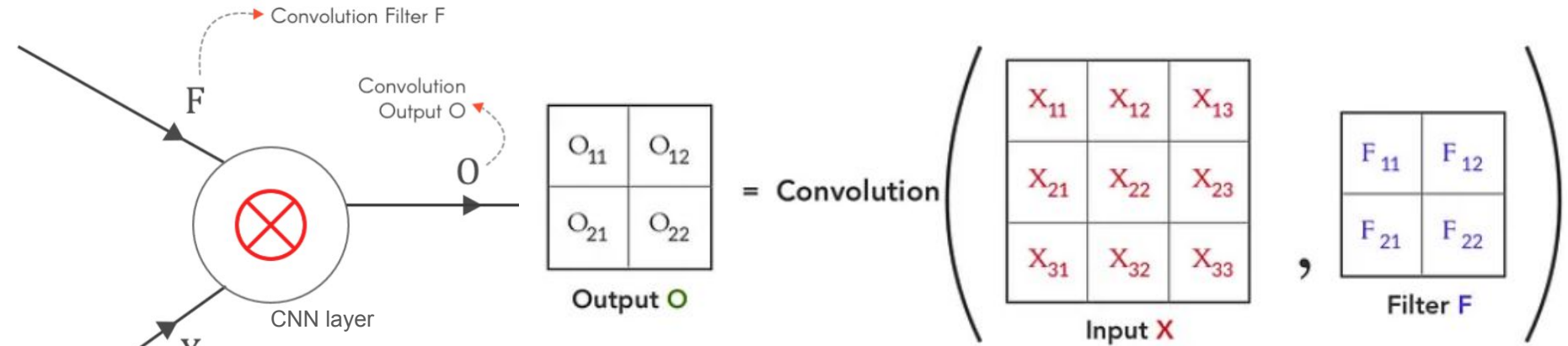


$\frac{\partial z}{\partial x}$ & $\frac{\partial z}{\partial y}$ are local gradients

$\frac{\partial L}{\partial z}$ is the loss from the previous layer which has to be backpropagated to other layers



Convolution example



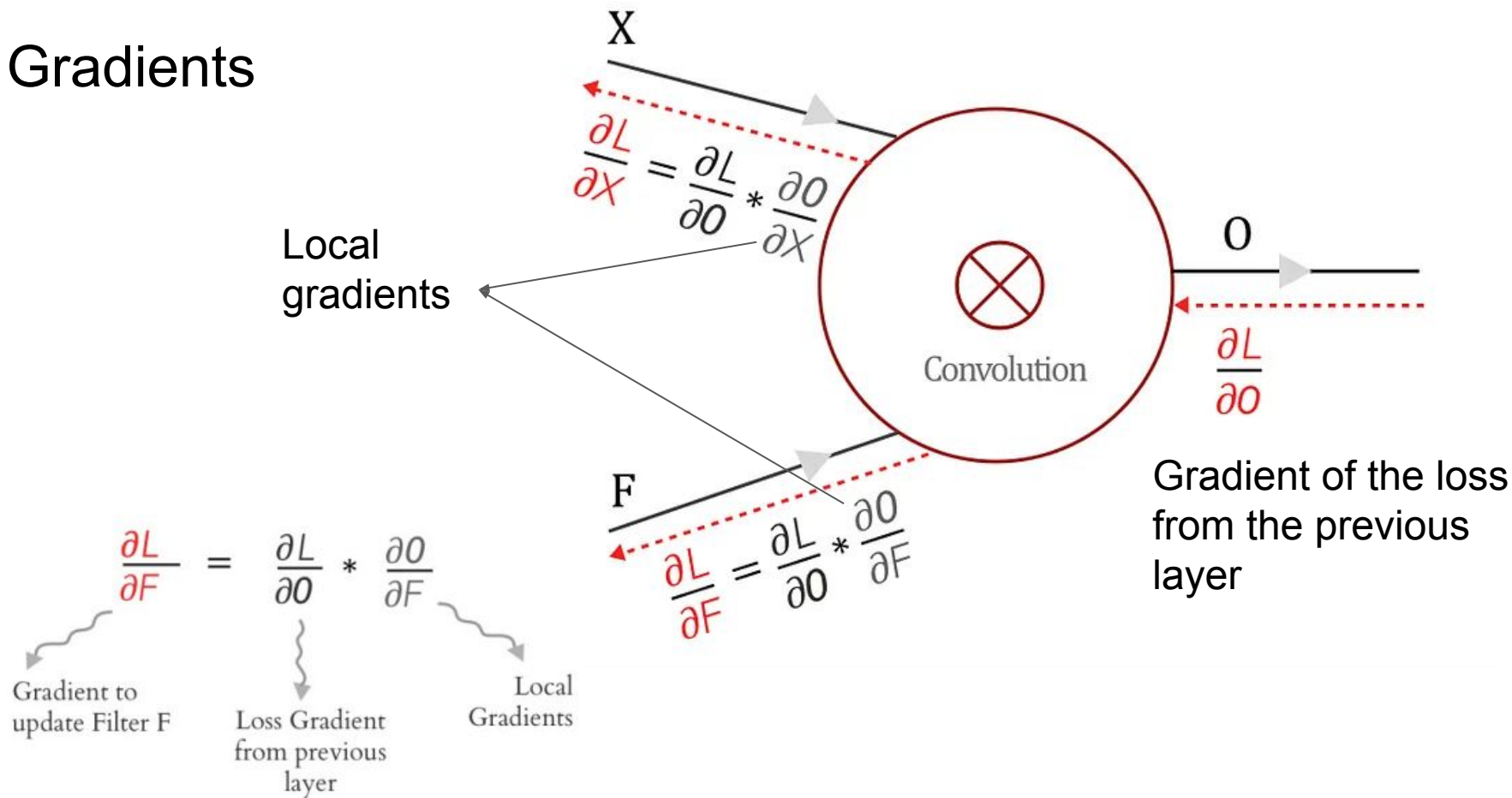
$$O_{11} = X_{11}F_{11} + X_{12}F_{12} + X_{21}F_{21} + X_{22}F_{22}$$

$$O_{12} = X_{12}F_{11} + X_{13}F_{12} + X_{22}F_{21} + X_{23}F_{22}$$

$$O_{21} = X_{21}F_{11} + X_{22}F_{12} + X_{31}F_{21} + X_{32}F_{22}$$

$$O_{22} = X_{22}F_{11} + X_{23}F_{12} + X_{32}F_{21} + X_{33}F_{22}$$

Gradients



Gradients to update filter/kernel

given $O_{11} = X_{11}F_{11} + X_{12}F_{12} + X_{21}F_{21} + X_{22}F_{22}$

Local gradients $\frac{\partial O_{11}}{\partial F_{11}} = X_{11} \quad \frac{\partial O_{11}}{\partial F_{12}} = X_{12} \quad \frac{\partial O_{11}}{\partial F_{21}} = X_{21} \quad \frac{\partial O_{11}}{\partial F_{22}} = X_{22}$

For every element of F

$$\frac{\partial L}{\partial F_i} = \sum_{k=1}^M \frac{\partial L}{\partial O_k} * \frac{\partial O_k}{\partial F_i}$$

Loss gradient for kth output
* local gradient of kth output

$$\begin{aligned} \frac{\partial L}{\partial F_{11}} &= \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{11}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{11}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{11}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{11}} \\ \frac{\partial L}{\partial F_{12}} &= \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{12}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{12}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{12}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{12}} \\ \frac{\partial L}{\partial F_{21}} &= \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{21}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{21}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{21}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{21}} \\ \frac{\partial L}{\partial F_{22}} &= \frac{\partial L}{\partial O_{11}} * \frac{\partial O_{11}}{\partial F_{22}} + \frac{\partial L}{\partial O_{12}} * \frac{\partial O_{12}}{\partial F_{22}} + \frac{\partial L}{\partial O_{21}} * \frac{\partial O_{21}}{\partial F_{22}} + \frac{\partial L}{\partial O_{22}} * \frac{\partial O_{22}}{\partial F_{22}} \end{aligned}$$

$$\begin{aligned} \frac{\partial L}{\partial F_{11}} &= \frac{\partial L}{\partial O_{11}} * X_{11} + \frac{\partial L}{\partial O_{12}} * X_{12} + \frac{\partial L}{\partial O_{21}} * X_{21} + \frac{\partial L}{\partial O_{22}} * X_{22} \\ \frac{\partial L}{\partial F_{12}} &= \frac{\partial L}{\partial O_{11}} * X_{12} + \frac{\partial L}{\partial O_{12}} * X_{13} + \frac{\partial L}{\partial O_{21}} * X_{22} + \frac{\partial L}{\partial O_{22}} * X_{23} \\ \frac{\partial L}{\partial F_{21}} &= \frac{\partial L}{\partial O_{11}} * X_{21} + \frac{\partial L}{\partial O_{12}} * X_{22} + \frac{\partial L}{\partial O_{21}} * X_{31} + \frac{\partial L}{\partial O_{22}} * X_{32} \\ \frac{\partial L}{\partial F_{22}} &= \frac{\partial L}{\partial O_{11}} * X_{22} + \frac{\partial L}{\partial O_{12}} * X_{23} + \frac{\partial L}{\partial O_{21}} * X_{32} + \frac{\partial L}{\partial O_{22}} * X_{33} \end{aligned}$$

Gradients to update filter

This is represented as a *convolution* operation between

input X and **loss gradient $\partial L / \partial \mathbf{O}$**

$$\begin{array}{|c|c|} \hline \frac{\partial L}{\partial F_{11}} & \frac{\partial L}{\partial F_{12}} \\ \hline \frac{\partial L}{\partial F_{21}} & \frac{\partial L}{\partial F_{22}} \\ \hline \end{array} = \text{Convolution} \left(\begin{array}{|c|c|c|} \hline X_{11} & X_{12} & X_{13} \\ \hline X_{21} & X_{22} & X_{23} \\ \hline X_{31} & X_{32} & X_{33} \\ \hline \end{array}, \begin{array}{|c|c|} \hline \frac{\partial L}{\partial O_{11}} & \frac{\partial L}{\partial O_{12}} \\ \hline \frac{\partial L}{\partial O_{21}} & \frac{\partial L}{\partial O_{22}} \\ \hline \end{array} \right)$$

where

$$\begin{array}{|c|c|c|} \hline X_{11} & X_{12} & X_{13} \\ \hline X_{21} & X_{22} & X_{23} \\ \hline X_{31} & X_{32} & X_{33} \\ \hline \end{array} = \text{Input X} \quad \begin{array}{|c|c|} \hline \frac{\partial L}{\partial O_{11}} & \frac{\partial L}{\partial O_{12}} \\ \hline \frac{\partial L}{\partial O_{21}} & \frac{\partial L}{\partial O_{22}} \\ \hline \end{array} = \frac{\partial L}{\partial \mathbf{O}} \quad \text{Loss gradient from previous layer}$$

$$\frac{\partial L}{\partial F_{11}} = \frac{\partial L}{\partial O_{11}} * X_{11} + \frac{\partial L}{\partial O_{12}} * X_{12} + \frac{\partial L}{\partial O_{21}} * X_{21} + \frac{\partial L}{\partial O_{22}} * X_{22}$$

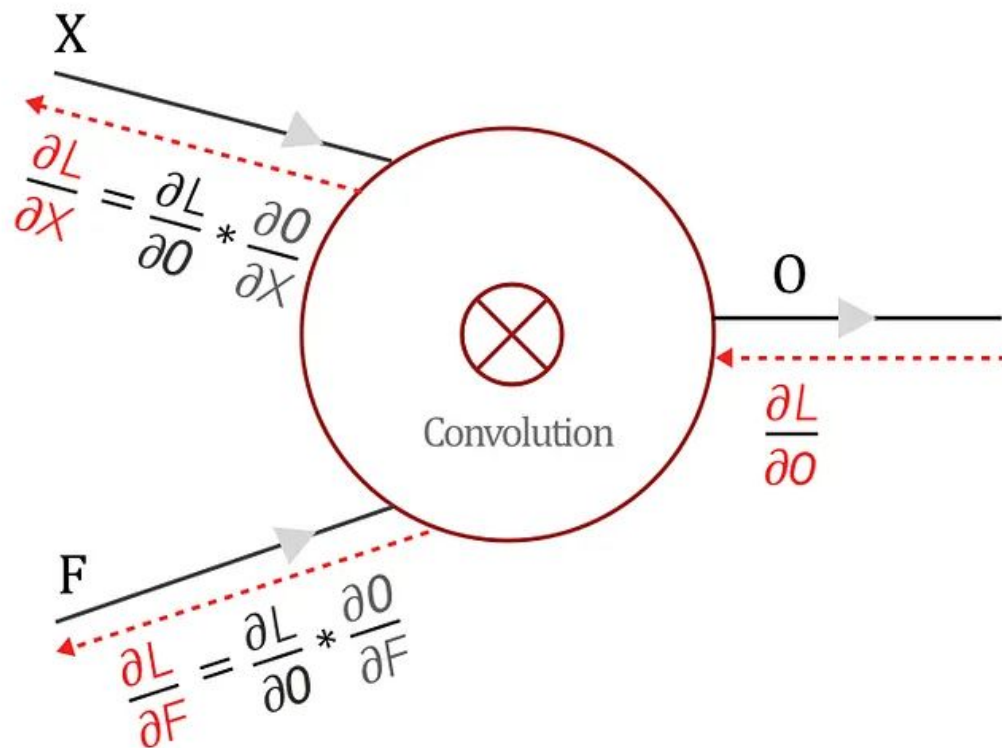
$$\frac{\partial L}{\partial F_{12}} = \frac{\partial L}{\partial O_{11}} * X_{12} + \frac{\partial L}{\partial O_{12}} * X_{13} + \frac{\partial L}{\partial O_{21}} * X_{22} + \frac{\partial L}{\partial O_{22}} * X_{23}$$

$$\frac{\partial L}{\partial F_{21}} = \frac{\partial L}{\partial O_{11}} * X_{21} + \frac{\partial L}{\partial O_{12}} * X_{22} + \frac{\partial L}{\partial O_{21}} * X_{31} + \frac{\partial L}{\partial O_{22}} * X_{32}$$

$$\frac{\partial L}{\partial F_{22}} = \frac{\partial L}{\partial O_{11}} * X_{22} + \frac{\partial L}{\partial O_{12}} * X_{23} + \frac{\partial L}{\partial O_{21}} * X_{32} + \frac{\partial L}{\partial O_{22}} * X_{33}$$

Gradients

$$\partial L / \partial X = ?$$



Finding $\partial L / \partial X$

given $O_{11} = X_{11}F_{11} + X_{12}F_{12} + X_{21}F_{21} + X_{22}F_{22}$

Local
gradients

$$\frac{\partial O_{11}}{\partial X_{11}} = F_{11} \quad \frac{\partial O_{11}}{\partial X_{12}} = F_{12} \quad \frac{\partial O_{11}}{\partial X_{21}} = F_{21} \quad \frac{\partial O_{11}}{\partial X_{22}} = F_{22}$$

For every element of X_i

$$\frac{\partial L}{\partial X_i} = \sum_{k=1}^M \frac{\partial L}{\partial O_k} * \frac{\partial O_k}{\partial X_i} \implies$$

$$\frac{\partial L}{\partial X_{11}} = \frac{\partial L}{\partial O_{11}} * F_{11}$$

$$\frac{\partial L}{\partial X_{12}} = \frac{\partial L}{\partial O_{11}} * F_{12} + \frac{\partial L}{\partial O_{12}} * F_{11}$$

$$\frac{\partial L}{\partial X_{13}} = \frac{\partial L}{\partial O_{12}} * F_{12}$$

$$\frac{\partial L}{\partial X_{21}} = \frac{\partial L}{\partial O_{11}} * F_{21} + \frac{\partial L}{\partial O_{21}} * F_{11}$$

$$\frac{\partial L}{\partial X_{22}} = \frac{\partial L}{\partial O_{11}} * F_{22} + \frac{\partial L}{\partial O_{12}} * F_{21} + \frac{\partial L}{\partial O_{21}} * F_{12} + \frac{\partial L}{\partial O_{22}} * F_{11}$$

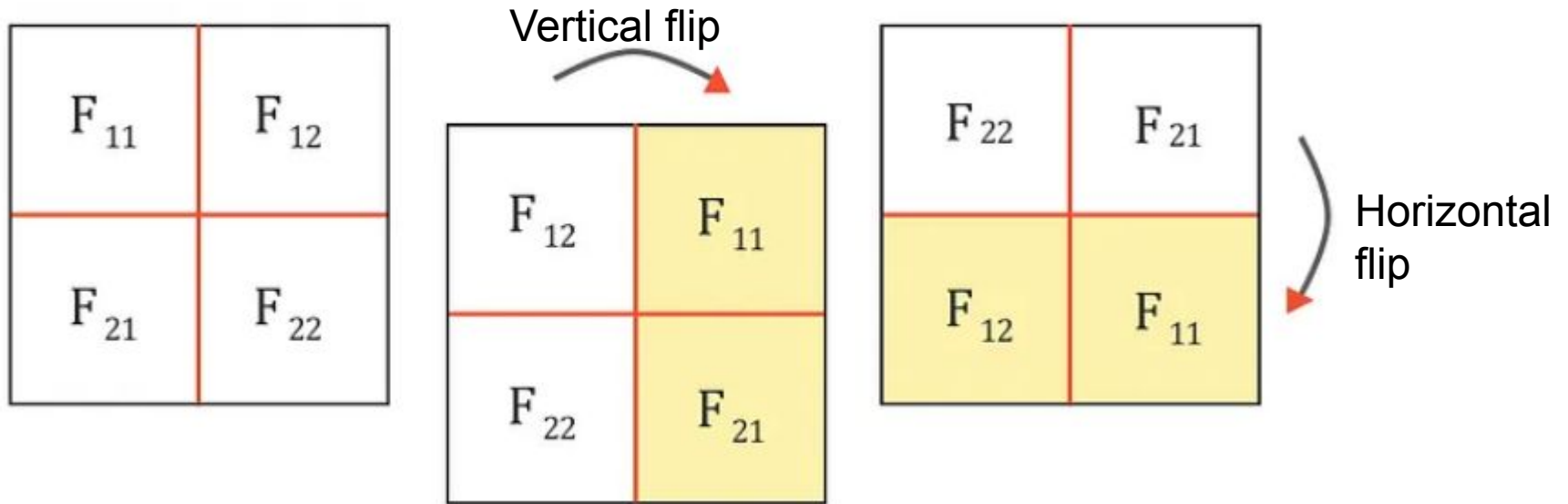
$$\frac{\partial L}{\partial X_{23}} = \frac{\partial L}{\partial O_{12}} * F_{22} + \frac{\partial L}{\partial O_{22}} * F_{12}$$

$$\frac{\partial L}{\partial X_{31}} = \frac{\partial L}{\partial O_{21}} * F_{21}$$

$$\frac{\partial L}{\partial X_{32}} = \frac{\partial L}{\partial O_{21}} * F_{22} + \frac{\partial L}{\partial O_{22}} * F_{21}$$

$$\frac{\partial L}{\partial X_{33}} = \frac{\partial L}{\partial O_{22}} * F_{22}$$

Gradients of input as convolution



Input gradient computation with flipped filter and full convolution

F_{22}	F_{21}
F_{12}	F_{11}

Filter F

$\frac{\partial L}{\partial O_{11}}$	$\frac{\partial L}{\partial O_{12}}$
$\frac{\partial L}{\partial O_{21}}$	$\frac{\partial L}{\partial O_{22}}$

Loss Gradient $\frac{\partial L}{\partial O}$

$$\frac{\partial L}{\partial X_{11}} = F_{11} * \frac{\partial L}{\partial O_{11}}$$

F_{22}	F_{21}	
F_{12}	$F_{11} \frac{\partial L}{\partial O_{11}}$	$\frac{\partial L}{\partial O_{12}}$
	$\frac{\partial L}{\partial O_{21}}$	$\frac{\partial L}{\partial O_{22}}$

Bottom to top &
Right to left

Input gradient computation with flipped filter and full convolution

F_{22}	F_{21}
F_{12}	F_{11}

Filter F

$\frac{\partial L}{\partial \theta_{11}}$	$\frac{\partial L}{\partial \theta_{12}}$
$\frac{\partial L}{\partial \theta_{21}}$	$\frac{\partial L}{\partial \theta_{22}}$

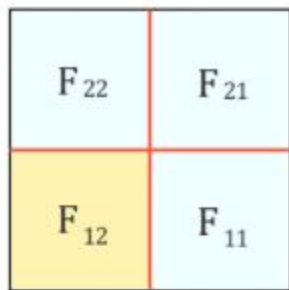
Loss Gradient $\frac{\partial L}{\partial \theta}$

$$\frac{\partial L}{\partial X_{12}} = F_{12} * \frac{\partial L}{\partial \theta_{11}} + F_{11} * \frac{\partial L}{\partial \theta_{12}}$$

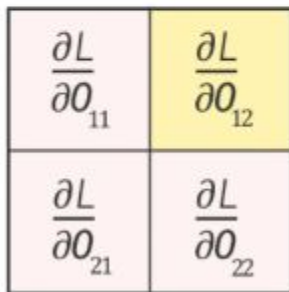
F_{22}	F_{21}
$F_{12} \frac{\partial L}{\partial \theta_{11}}$	$F_{11} \frac{\partial L}{\partial \theta_{12}}$
$\frac{\partial L}{\partial \theta_{21}}$	$\frac{\partial L}{\partial \theta_{22}}$

Bottom to top &
Right to left

Input gradient computation with flipped filter and full convolution

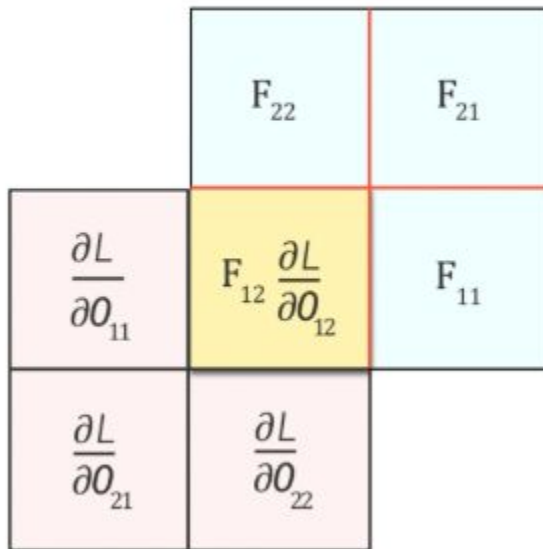


Filter F



Loss Gradient $\frac{\partial L}{\partial O}$

$$\frac{\partial L}{\partial X_{13}} = F_{12} * \frac{\partial L}{\partial O_{12}}$$



Bottom to top &
Right to left

Input gradient computation with flipped filter and full convolution

F_{22}	F_{21}
F_{12}	F_{11}

Filter F

$\frac{\partial L}{\partial O_{11}}$	$\frac{\partial L}{\partial O_{12}}$
$\frac{\partial L}{\partial O_{21}}$	$\frac{\partial L}{\partial O_{22}}$

Loss Gradient $\frac{\partial L}{\partial O}$

$$\frac{\partial L}{\partial X_{21}} = F_{21} * \frac{\partial L}{\partial O_{11}} + F_{11} * \frac{\partial L}{\partial O_{21}}$$

F_{22}	$F_{21} \frac{\partial L}{\partial O_{11}}$	$\frac{\partial L}{\partial O_{12}}$
F_{12}	$F_{11} \frac{\partial L}{\partial O_{21}}$	$\frac{\partial L}{\partial O_{22}}$

Bottom to top &
Right to left

Input gradient computation with flipped filter and full convolution

F_{22}	F_{21}
F_{12}	F_{11}

Filter F

$\frac{\partial L}{\partial \theta_{11}}$	$\frac{\partial L}{\partial \theta_{12}}$
$\frac{\partial L}{\partial \theta_{21}}$	$\frac{\partial L}{\partial \theta_{22}}$

Loss Gradient $\frac{\partial L}{\partial \theta}$

$$\frac{\partial L}{\partial x_{22}} = F_{22} * \frac{\partial L}{\partial \theta_{11}} + F_{21} * \frac{\partial L}{\partial \theta_{12}} + F_{12} * \frac{\partial L}{\partial \theta_{21}} + F_{11} * \frac{\partial L}{\partial \theta_{22}}$$

$F_{22} \frac{\partial L}{\partial \theta_{11}}$	$F_{21} \frac{\partial L}{\partial \theta_{12}}$
$F_{12} \frac{\partial L}{\partial \theta_{21}}$	$F_{11} \frac{\partial L}{\partial \theta_{22}}$

Bottom to top &
Right to left

Input gradient computation with flipped filter and full convolution

F_{22}	F_{21}
F_{12}	F_{11}

Filter F

$$\frac{\partial L}{\partial X_{23}} = F_{22} * \frac{\partial L}{\partial \theta_{12}} + F_{12} * \frac{\partial L}{\partial \theta_{22}}$$

$\frac{\partial L}{\partial \theta_{11}}$	$F_{22} \frac{\partial L}{\partial \theta_{12}}$	F_{21}
$\frac{\partial L}{\partial \theta_{21}}$	$F_{12} \frac{\partial L}{\partial \theta_{22}}$	F_{11}

$\frac{\partial L}{\partial \theta_{11}}$	$\frac{\partial L}{\partial \theta_{12}}$
$\frac{\partial L}{\partial \theta_{21}}$	$\frac{\partial L}{\partial \theta_{22}}$

Loss Gradient $\frac{\partial L}{\partial \theta}$

Bottom to top &
Right to left

Input gradient computation with flipped filter and full convolution

F_{22}	F_{21}
F_{12}	F_{11}

Filter F

$\frac{\partial L}{\partial o_{11}}$	$\frac{\partial L}{\partial o_{12}}$
$\frac{\partial L}{\partial o_{21}}$	$\frac{\partial L}{\partial o_{22}}$

Loss Gradient $\frac{\partial L}{\partial o}$

	$\frac{\partial L}{\partial o_{11}}$	$\frac{\partial L}{\partial o_{12}}$
F_{22}	$F_{21} \frac{\partial L}{\partial o_{21}}$	$\frac{\partial L}{\partial o_{22}}$
F_{12}	F_{11}	

$$\frac{\partial L}{\partial x_{31}} = F_{21} * \frac{\partial L}{\partial o_{21}}$$

Bottom to top &
Right to left

Input gradient computation with flipped filter and full convolution

F_{22}	F_{21}
F_{12}	F_{11}

Filter F

$\frac{\partial L}{\partial o_{11}}$	$\frac{\partial L}{\partial o_{12}}$
$\frac{\partial L}{\partial o_{21}}$	$\frac{\partial L}{\partial o_{22}}$

Loss Gradient $\frac{\partial L}{\partial o}$

$\frac{\partial L}{\partial o_{11}}$	$\frac{\partial L}{\partial o_{12}}$
$F_{22} \frac{\partial L}{\partial o_{21}}$	$F_{21} \frac{\partial L}{\partial o_{22}}$
F_{12}	F_{11}

$$\frac{\partial L}{\partial x_{32}} = F_{22} * \frac{\partial L}{\partial o_{21}} + F_{21} * \frac{\partial L}{\partial o_{22}}$$

Bottom to top &
Right to left

Input gradient computation with flipped filter and full convolution

F_{22}	F_{21}
F_{12}	F_{11}

Filter F

$\frac{\partial L}{\partial o_{11}}$	$\frac{\partial L}{\partial o_{12}}$
$\frac{\partial L}{\partial o_{21}}$	$\frac{\partial L}{\partial o_{22}}$

Loss Gradient $\frac{\partial L}{\partial o}$

$\frac{\partial L}{\partial o_{11}}$	$\frac{\partial L}{\partial o_{12}}$	
$\frac{\partial L}{\partial o_{21}}$	$F_{22} \frac{\partial L}{\partial o_{22}}$	F_{21}
	F_{12}	F_{11}

$$\frac{\partial L}{\partial x_{33}} = F_{22} * \frac{\partial L}{\partial o_{22}}$$

Bottom to top &
Right to left

Input gradient computation with flipped filter and full convolution

$$\begin{array}{|c|c|c|} \hline \frac{\partial L}{\partial X_{11}} & \frac{\partial L}{\partial X_{12}} & \frac{\partial L}{\partial X_{13}} \\ \hline \frac{\partial L}{\partial X_{21}} & \frac{\partial L}{\partial X_{22}} & \frac{\partial L}{\partial X_{23}} \\ \hline \frac{\partial L}{\partial X_{31}} & \frac{\partial L}{\partial X_{32}} & \frac{\partial L}{\partial X_{33}} \\ \hline \end{array} = \text{Full Convolution} \left(\begin{array}{|c|c|} \hline F_{22} & F_{21} \\ \hline F_{12} & F_{11} \\ \hline \end{array}, \begin{array}{|c|c|} \hline \frac{\partial L}{\partial O_{11}} & \frac{\partial L}{\partial O_{12}} \\ \hline \frac{\partial L}{\partial O_{21}} & \frac{\partial L}{\partial O_{22}} \\ \hline \end{array} \right)$$

$\frac{\partial L}{\partial X}$
Filter F
Loss Gradient $\frac{\partial L}{\partial O}$

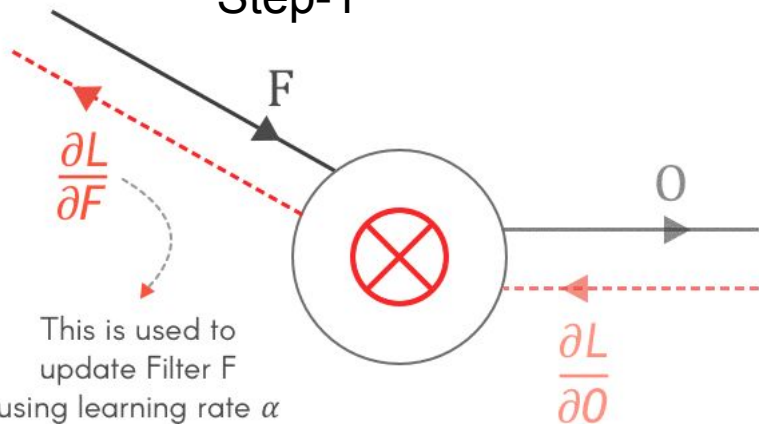
$$\frac{\partial L}{\partial X} = \text{Full Convolution} \left(\begin{array}{c} 180^\circ \text{rotated} \\ \text{Filter } F \end{array}, \begin{array}{c} \text{Loss} \\ \text{Gradient } \frac{\partial L}{\partial O} \end{array} \right)$$

Full convolution: slide filter from Bottom to top & Right to left

Summary

- Forward pass the input,
- Find the output layer gradient and
- Repeat following steps for each layer

Step-1

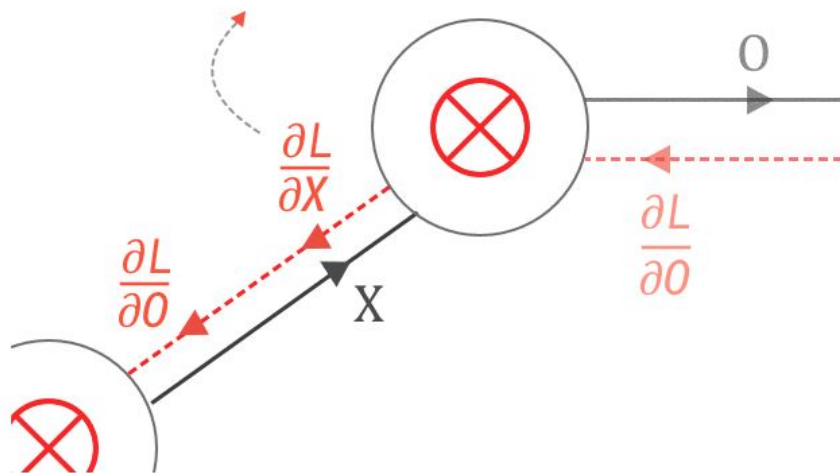


$$F_{\text{updated}} = F - \alpha \frac{\partial L}{\partial F}$$

$$\frac{\partial L}{\partial F} = \text{Convolution} \left(\text{Input } X, \text{ Loss gradient } \frac{\partial L}{\partial O} \right)$$

Step-2

Since X is the output of the previous layer, $\partial L / \partial X$ becomes the loss gradient for the previous layer



$$\frac{\partial L}{\partial X} = \text{Full Convolution} \left(180^\circ \text{ rotated Filter } F, \text{ Loss Gradient } \frac{\partial L}{\partial O} \right)$$

Both the Forward pass and the Backpropagation of a Convolutional layer are Convolutions